Rockchip Sensor驱动调试开发指南

文件标识：RK-KF-YF-A56

发布版本：V1.1.0

日　　期：2025-09-28

文件密级：□绝密　□秘密　□内部资料　■公开

免责声明

本文档按"现状"提供，瑞芯微电子股份有限公司（"本公司"，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

商标声明

"Rockchip"、"瑞芯微"、"瑞芯"均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自拥有者所有。

版权所有 © 2025 瑞芯微电子股份有限公司

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址：　福建省福州市铜盘路软件园A区18号

网址：　www.rock-chips.com

客户服务电话：+86-4007-700-590

客户服务传真：+86-591-83951833

客户服务邮箱：fae@rock-chips.com

前言

概述

本文档主要介绍 Rockchip Linux 平台，Camera 驱动调试与功能开发流程、常见问题及解决方案。

产品版本

芯片名称	内核版本
RV1109/RV1126	Linux 4.19
RK356X	Linux 4.19
RK3588	Linux 5.10
RK3576	Linux5.10
RV1106 / RV1103	Linux5.10
RV1106B / RV1103B	Linux5.10
RV1126B	Linux 6.1

读者对象

本文档（本指南）主要适用于以下工程师：

技术支持工程师

软件开发工程师

修订记录

版本号	作者	修改日期	修改说明
V1.0.0	MLC	2025-08-10	初始版本
V1.1.0	SYF	2025-09-28	增加MCU 快启配置指南

1. 目录

Rockchip Sensor驱动调试开发指南

1. 目录
2. 调试前的准备
   - 2.1 确认主控规格
   - 2.2 Sensor DataSheet
   - 2.3 Sensor Initialize Settings
3. 驱动开发
   - 3.1 驱动代码编写
     - 3.1.1 Sensor 重要参数
     - 3.1.2 Sensor 重要寄存器
       - 3.1.2.1 **Chip id**
       - 3.1.2.2 **Ctrol mode**
       - 3.1.2.3 **Exposure reg**
       - 3.1.2.4 **Gain reg**
       - 3.1.2.5 Group hold
       - 3.1.2.6 Test pattern
         - 3.1.2.6.1 Test Pattern的用途和作用
         - 3.1.2.6.2 **常见的 Test Pattern 类型**
       - 3.1.2.7 VTS
       - 3.1.2.8 Mirror / Flip
   - 3.2 驱动框架与说明
     - 3.2.1 数据类型说明
       - 3.2.1.1 struct i2c_driver
       - 3.2.1.2 struct v4l2_subdev_ops
       - 3.2.1.3 struct v4l2_subdev_core_ops
       - 3.2.1.4 struct v4l2_subdev_video_ops
       - 3.2.1.5 struct v4l2_subdev_pad_ops
       - 3.2.1.6 struct v4l2_ctrl_ops
     - 3.2.2 API介绍
       - 3.2.2.1 xxxx_set_fmt
       - 3.2.2.2 xxxx_get_fmt
       - 3.2.2.3 xxxx_enum_mbus_code
       - 3.2.2.4 xxxx_enum_frame_sizes
       - 3.2.2.5 xxxx_g_frame_interval
       - 3.2.2.6 xxxx_s_stream
       - 3.2.2.7 xxxx_runtime_resume
       - 3.2.2.8 xxxx_runtime_suspend
       - 3.2.2.9 xxxx_set_ctrl
       - 3.2.2.10 xxx_enum_frame_interval
       - 3.2.2.11 xxxx_g_mbus_config
       - 3.2.2.12 xxxx_get_selection
       - 3.2.2.13 xxxx_ioctl
   - 3.3 驱动移植步骤
     - 3.3.1 实现标准I2C子设备驱动部分
     - 3.3.2 实现v4l2子设备驱动
   - 3.4 Makefile修改
   - 3.5 Kconfig修改

4. 快速启动
    4.1 概念解释
    4.2 驱动相关
        4.2.1 内核驱动
        4.2.2 MCU驱动
    4.3 MCU 快启配置指南
        4.3.1 驱动框架与说明
            4.3.1.1 数据类型说明
                4.3.1.1.1 struct rk_camera_ops
                4.3.1.1.2 struct xxx_mode
                4.3.1.1.3 struct xxx_dev
            4.3.1.2 API描述
                4.3.1.2.1 rk_xxx_control
                4.3.1.2.2 rk_xxx_init
                4.3.1.2.3 xxx_stream_on
                4.3.1.2.4 xxx_stream_off
                4.3.1.2.5 rk_xxx_get_expinf
                4.3.1.2.6 rk_xxx_set_expval
                4.3.1.2.7 rk_xxx_set_vts
                4.3.1.2.8 rk_xxx_get_intput_fmt
                4.3.1.2.9 rk_xxx_set_intput_fmt
                4.3.1.2.10 rk_xxx_match_dst_config
                4.3.1.2.11 rk_xxx_set_flip_mirror
                4.3.1.2.12 xxx_stream_on_late
                4.3.1.2.13 xxx_quick_stream
                4.3.1.2.14 rk_camera_xxx_init
        4.3.2 摄像头驱动移植
            4.3.2.1 添加编译配置选项
            4.3.2.2 寄存器序列转换
                4.3.2.2.1 序列转换步骤
            4.3.2.3 配置基础函数
        4.3.3 MCU板级配置说明
            4.3.3.1 CIS(cmos image sensor)设备注册
                4.3.3.1.1 struct gpio_desc
                4.3.3.1.2 camera_board_desc 结构体说明
                4.3.3.1.3 cif_hw_config 函数说明
                4.3.3.1.4 单摄注册
        4.3.4 如何新增板级配置
5. 新的功能支持
    5.1 AOV
        5.1.1 场景说明
        5.1.2 工作流程
        5.1.3 概念说明
            5.1.3.1 硬件 standby 模式
            5.1.3.2 软件 standby 模式
        5.1.4 驱动实现
        5.1.5 参考文档
    5.2 同步模式 (主从模式)
        5.2.1 Sensor 的 Slave模式
        5.2.2 驱动实现
    5.3 闪光灯控制
        5.3.1 驱动实现
        5.3.2 参考文档

- 5.4 分辨率切换
  - 5.4.1 驱动实现
- 5.5 多 lane 配置的兼容
  - 5.5.1 驱动实现
- 5.6 其他

6. 常见问题分析与解决方案
   - 6.1 Sensor不识别，probe 失败，I2C通讯失败
     - 6.1.1 测量不到 MCLK 和 VDD 电压
     - 6.1.2 检查 Sensor 的上电时序是否满足要求
   - 6.2 驱动中的 `exp_def` 、 `hts_def` 、 `vts_def` 默认值应该设多少
   - 6.3 `link_freq` 与 `pixel_rate` 值应该是多少
   - 6.4 怎么才算点亮 Sensor
   - 6.5 Sensor 可以出流，报 CRC、ECC 等相关错误
     - 6.5.1 **确认错误来源和日志细节**
     - 6.5.2 **软件配置检查 (驱动与DTS)**
     - 6.5.3 **硬件链路排查 (物理层问题占90%以上！)**
     - 6.5.4 **系统级验证与调试**
     - 6.5.5 **紧急情况处理策略**
     - 6.5.6 **典型修复措施**
     - 6.5.7 Sensor index注意事项
7. 可参考的文档
8. 附录
   - 8.1 CIS驱动V4L2-controls列表
   - 8.2 MEDIA_BUS_FMT列表
   - 8.3 Camera 参考驱动列表

2. 调试前的准备

Camera 驱动调试前需准备以下资料：

2.1 确认主控规格

确认当前 Sensor 所使用的主控平台支持的最大分辨率、输入频率上限、最高每 Lane 的速率、支持的Lane 数、数据输入的接口类型等。

2.2 Sensor DataSheet

找 Sensor 原厂提供 相关Sensor Datasheet，方便查阅相关信息：

- 确认硬件各路电压、上下电时序。
- 确认 Sensor Chip ID，Sensor probe 时 check 使用。
- 确认曝光时间、增益如何设置，帧率如何修改。
- 确认是否支持 Slave 模式，多摄如何连接、如何配置相关寄存器。
- 测试模式是否支持。
- 确认是否支持高动态模式。
- 确认 Mirror / Flip 如何配置。

2.3 Sensor Initialize Settings

获取 Sensor 寄存器配置列表。

一般由 Sensor 原厂提供。明确具体需求，如分辨率、帧率、lane 数、每lane 速率、VTS、HTS、位宽、主时钟频率等。

一般至少准备标准分辨率序列。根据实际项目需求，确定是否需要准备binning、crop等类型的特殊分辨率序列。

例如思特威的寄存器配置文件 cleaned_0x2b_SC530AI_27Minput_1080Mbps_2lane_10bit_1920x1080_60fps.ini，文件名中会给出相关信息：

Sensor 名：SC530AI

主控提供给 Sensor 的MCLK：27Mhz
Sensor 输入给主控的 MIPI 速率为每 Lane： 1080Mbps
使用Lane数: 2
位宽：10bit
分辨率：1920x1080（1080p）
帧率：60pfs

下面给出向 Sensor 厂家申请配置模板：具体条目根据项目实际情况选择。

```
sensor: XXXX
mclk: 24MHz/27Mhz/37.5Mhz
mode: raw8/raw10/raw12/raw16
output:720p/1080p/1440p/...(分辨率)
接口: mipi 1/2/4 Lane
帧率: 30/60/120 fps
主控平台: RK3588/RV1106/RV1103B/RV1126B/...
项目(产品)形态: 门铃/门铃/IPC/... （可选）
```

3. 驱动开发

3.1 驱动代码编写

编写新的 Sensor 驱动时，可基于一款规格接近的 Sensor （`kernel/drivers/media/i2c/`）驱动或相同厂家型号相近的驱动修改 。

以下章节均以思特威 SC200AI Sensor 驱动开发为例，说明 RK 平台 Sensor 驱动开发的流程。

3.1.1 Sensor 重要参数

Sensor mode 结构体定义了 Sensor 的重要参数，不同分辨率都需填充准确的参数信息。

```
struct XXXX_mode {
    u32 bus_fmt;    //输出像素格式
    u32 width;      //输出宽度
    u32 height;     //输出高度
    struct v4l2_fract max_fps; //最大输出帧率
    u32 hts_def;    //默认行总周期
    u32 vts_def;    //默认帧总周期
    u32 exp_def;    //默认曝光值
    u32 mipi_freq_idx;//链接频率索引值
    u32 bpp;        //每像素所用bit数
    const struct regval *reg_list; //寄存器列表
    u32 hdr_mode;   //宽动态模式
    u32 vc[PAD_MAX];//虚拟通道数组
};
```

supported_modes 结构体数组保存了支持的不同分辨率的 mode 结构体，用来在 probe 初始化时或切换分辨率时遍历、匹配。

以 SC200AI 为例：

```
static const struct sc200ai_mode supported_modes[] = {
    { // 1080p 30fps 线性配置参数
```

```
        .width = 1920,
        .height = 1080,
        .max_fps = {
            .numerator = 10000,
            .denominator = 300000,
        },
        .exp_def = 0x0080,
        .hts_def = 0x44C * 2,
        .vts_def = 0x0465,
        .bus_fmt = MEDIA_BUS_FMT_SBGGR10_1X10,
        .reg_list = sc200ai_linear_10_1920x1080_30fps_regs,
        .hdr_mode = NO_HDR,
        .bpp = 10,
        .mipi_freq_idx = 1,
        .vc[PAD0] = V4L2_MBUS_CSI2_CHANNEL_0,
    },
    { // 540p 120fps 线性配置参数
        .width = 960,
        .height = 540,
        .max_fps = {
            .numerator = 10000,
            .denominator = 1200000,
        },
        .exp_def = 0x0080,
        .hts_def = 0x420,
        .vts_def = 0x249,
        .bus_fmt = MEDIA_BUS_FMT_SBGGR10_1X10,
        .reg_list = sc200ai_linear_10_960x540_120fps_regs,
        .hdr_mode = NO_HDR,
        .bpp = 10,
        .mipi_freq_idx = 0,
        .vc[PAD0] = V4L2_MBUS_CSI2_CHANNEL_0,
    },
    { // 1080p 30fps 宽动态配置参数
        .width = 1920,
        .height = 1080,
        .max_fps = {
            .numerator = 10000,
            .denominator = 300000,
        },
        .exp_def = 0x0080,
        .hts_def = 0x44C * 2,
        .vts_def = 0x08CC,
        .bus_fmt = MEDIA_BUS_FMT_SBGGR10_1X10,
        .reg_list = sc200ai_hdr_10_1920x1080_regs,
        .hdr_mode = HDR_X2,
        .bpp = 10,
        .mipi_freq_idx = 1,
        .vc[PAD0] = V4L2_MBUS_CSI2_CHANNEL_1,
        .vc[PAD1] = V4L2_MBUS_CSI2_CHANNEL_0,//L->csi wr0
        .vc[PAD2] = V4L2_MBUS_CSI2_CHANNEL_1,
        .vc[PAD3] = V4L2_MBUS_CSI2_CHANNEL_1,//M->csi wr2
    },
```

```
}
```

参数说明：

- bus_fmt是 Sensor 输出的像素排列格式，一般在 Sensor 给出的序列里面有多少bit(比如10bit)，BGGR 是像素排列顺序，即Bayer格式，Bayer顺序错了不影响出图(但会出现色偏)。
- width、height 是 Sensor 输出的有效宽、高，是最重要的参数，一般是小于 hts 和 vts。
- max_fps 是当前 mode支持输出的最大帧率，是用 denominato / numerator 为帧率，一般 numerator 不改，如果是准确的 30 帧，denominator 就是 300000。
- exp_def 默认的曝光值（根据初始化寄存器定义的值填写）。
- hts_def 默认一行的总点数（根据初始化寄存器定义的值填写）。
- vts_def 默认一帧的总行数（根据初始化寄存器定义的值填写），此值不要写错，否则会影响输出帧率。
- global_reg_list 是 Sensor 的通用初始化寄存器，若没有，可以为空。
- reg_list 是 Sensor 厂商给的寄存器序列。
- hdr_mode 是对应当前参数的hdr模式，NO_HDR 表示线性模式，HDR_X2 / HDR_X3 表示宽动态，使用2帧或3帧融合。
- link_freq_idx 对应下面 link_freq_menu_items 数组中的频率索引，一般厂家给的是每 Lane 速率Mbps, 因为 MIPI 双边沿采样的原因，MIPI 频率是采样率的一半，以第一组线性为例，厂家给的序列里写的速率是 720Mbps， MIPI 频率则是 360M，也就是 link_freq 链接频率是 360M。
  重要提示：这个参数不对容易导致收图失败，要再三确认。
- bpp 是表明每个像素占用的bit位数。

注意：

线性与宽动态的配置区别有两处，一是 hdr_mode 的不同，二是 vc 虚拟通道数组的不同。

- 线性：
  hdr_mode = NO_HDR，
  vc[PAD0] = V4L2_MBUS_CSI2_CHANNEL_0；
- 宽动态：
  hdr_mode = HDR_X2/HDR_X3，
  vc[PAD0] = V4L2_MBUS_CSI2_CHANNEL_0；
  vc[PAD1] = V4L2_MBUS_CSI2_CHANNEL_0;
  vc[PAD2] = V4L2_MBUS_CSI2_CHANNEL_1;
  vc[PAD3] = V4L2_MBUS_CSI2_CHANNEL_1;

3.1.2 Sensor 重要寄存器

以下简单罗列并介绍了 Sensor 驱动开发中可能使用到的一些重要寄存器，根据 Sensor 厂家、型号不同，可能存在一些差异。

3.1.2.1 Chip id

Sensor 芯片ID，是每款 Sensor 的身份证，用来在驱动加载时识别 Sensor 是否被正确加载。Sensor probe接口中会check每款芯片的 Chip id，若读取到该 Sensor Chip id，则认为 Sensor 设备识别正常，I2C 读写通讯正常，各路电压正常。

```
#define CHIP_ID                 0xcb1c
#define SC200AI_REG_CHIP_ID     0x3107
```

注意：

有些Sensor 没有Chip id，无法验证 Sensor 设备是否正常识别和挂载时，可以选取某个（某些个）只读寄存器读取，验证读取结果是否是默认值来确认。

3.1.2.2 Ctrol mode

Sensor 控制起流、停流的控制寄存器，一般等 Sensor 所有寄存器都写入成功后，写入该寄存器可以控制 Sensor 出流或停流。

```
#define SC200AI_REG_CTRL_MODE       0x0100
#define SC200AI_MODE_SW_STANDBY     0x0
#define SC200AI_MODE_STREAMING      BIT(0)
```

注意：

请不要在寄存器序列中配置该寄存器的值，我们使用起流和停流的接口调用单独控制该寄存器的写入。

3.1.2.3 Exposure reg

曝光控制寄存器，根据线性/宽动态模式，可能会配置不同的曝光控制寄存器，具体根据 Sensor 手册中说明。

```
#define SC200AI_REG_EXPOSURE_H      0x3e00  //长曝光最高位
#define SC200AI_REG_EXPOSURE_M      0x3e01  //长曝光中间位
#define SC200AI_REG_EXPOSURE_L      0x3e02  //长曝光最低位
#define SC200AI_REG_SEXPOSURE_H     0x3e22  //短曝光最高位
#define SC200AI_REG_SEXPOSURE_M     0x3e04  //短曝光中间位
#define SC200AI_REG_SEXPOSURE_L     0x3e05  //短曝光最低位
#define SC200AI_EXPOSURE_MIN        1       //最新曝光行，一般不改，除非sensor有最小曝光限制
#define SC200AI_EXPOSURE_STEP       1       //曝光步进，一般不用修改
```

注意：

- 有些 Sensor 的曝光寄存器可能不止 16bit 的（比如高字节+低字节，如 这里是 24bit，高、中、低三个字节），需要分多次写入。也有可能是长帧或短帧的，需要根据实际情况写入。
- 增益值可能需要按照特定公式计算，比如：
  - 模拟增益可能是线性值，也可能是查表；
  - 数字增益可能是浮点倍数，但写入的是定点数（如 Q8.8 格式）

3.1.2.4 Gain reg

增益控制寄存器，分为 模拟增益控制寄存器和数字增益控制寄存器。

每种增益控制寄存器有时分主增益（Coarse gain， 粗增益，大粒度的，一般对应的是增益的整数部分，用来对增益的粗调）和精细增益（Fine gain，精增益，细粒度的，一般对应的是增益的小数部分，用来对增益的微调）。

下面简单介绍下相关概念：

模拟增益（Analog Gain）

- 是在 模拟信号域（ADC 之前） 对传感器输出的电压信号进行放大；
- 优点：对图像噪声影响相对小一些（相比数字增益）；
- 缺点：放大倍数有限，过高会引入更多噪声和非线性失真；
- 对应寄存器：`ANA_GAIN`（主）、`ANA_FINE_GAIN`（微调）

数字增益（Digital Gain）

- 是在 数字信号域（ADC 之后） 对图像的原始数字值（比如 10bit / 12bit 数据）进行乘法放大；
- 优点：实现简单，灵活；
- 缺点：本质是对数字值做乘法，**会放大噪声和量化误差**，尤其在低光下容易产生彩色噪点；
- 对应寄存器：`DIG_GAIN`（主）、`DIG_FINE_GAIN`（微调）

增益调节的目标是什么？

- 在 低光环境 下提升画面亮度（通过提高增益）
- 在自动曝光（AE）算法中，**动态调整增益与曝光时间（shutter/line count）** 来达到目标亮度
- 控制 **图像信噪比（SNR）、动态范围** 等

```
#define SC200AI_REG_DIG_GAIN        0x3e06  //数字主增益
#define SC200AI_REG_DIG_FINE_GAIN   0x3e07  //数组精增益
#define SC200AI_REG_ANA_GAIN        0x3e08  //模拟主增益
#define SC200AI_REG_ANA_FINE_GAIN   0x3e09  //模拟精增益
#define SC200AI_REG_SDIG_GAIN       0x3e10  //短帧的数字主增益
#define SC200AI_REG_SDIG_FINE_GAIN  0x3e11  //短帧的数组精增益
#define SC200AI_REG_SANA_GAIN       0x3e12  //短帧的模拟主增益
#define SC200AI_REG_SANA_FINE_GAIN  0x3e13  //短帧的模拟精增益
#define SC200AI_GAIN_MIN            0x0040  //增益最小值
#define SC200AI_GAIN_MAX            (54 * 32 * 64)       //53.975*31.75*64 增益最大值
#define SC200AI_GAIN_STEP           1   //增益步进
#define SC200AI_GAIN_DEFAULT        0x0800  //默认增益值
#define SC200AI_LGAIN               0   //长帧增益
#define SC200AI_SGAIN               1   //短帧增益
```

注意：

在支持 HDR 高动态模式时，设置 Sensor 增益的时候，可能需要设置长帧和短帧增益，此时定义了 SC200AI_LGAIN、SC200AI_LGAIN 来区分。

3.1.2.5 Group hold

Group Hold（组保持 / 群组保持） 是 CMOS 图像传感器中用于 **控制曝光（shutter）、增益等参数同步更新** 的一种机制，主要目的是：

> 在 **同一帧图像的采集过程中，保证所有相关参数（如模拟增益、数字增益、行曝光时间等）保持一致，避免参数中途变化导致图像出现异常（如竖纹、明暗不均、部分过曝/欠曝等）。**

具体来说：在 CMOS 传感器中，很多关键参数（比如曝光时间、模拟/数字增益等）是通过寄存器写入来配置的；

但这些寄存器的修改，**通常并不会立即生效**，而是要等到 **下一个帧/行周期开始时才生效**；如果你在 **一帧图像的采集过程中，修改了这些参数**，就可能导致 **图像的一部分用旧参数、另一部分用新参数**，出现明显的分界线或异常；
Group Hold 就是用来"冻结"这些参数的更新时机"，让它们在指定的时刻统一生效，从而保证一帧图像内参数的一致性。

```
#define SC200AI_REG_GROUP_HOLD      0x3812 //group hold 控制寄存器
#define SC200AI_GROUP_HOLD_START    0x00  //开启Group Hold（冻结参数更新）即：Group Hold Start
#define SC200AI_GROUP_HOLD_END      0x30  //关闭Group Hold（释放参数更新)即：Group Hold End
```

3.1.2.6 Test pattern

测试图案寄存器，是指 **图像传感器（如 CMOS/CCD Sensor）在特定模式下，不采集真实的光学图像，而是直接输出由传感器内部生成的固定图案（如彩色条纹、渐变、棋盘格、单色块等）的一种功能。**

简单来说就是：让摄像头 **不拍真实场景，而是输出一个传感器内部生成的"假图像"**，用于测试、调试或验证目的。

这些图案通常是：

- 彩色条纹（Color Bars）
- 渐变色（Gradient）
- 棋盘格（Checkerboard）
- 单色（纯红 / 绿 / 蓝 / 黑 / 白）
- 网格（Grid）
- 固定图案噪声（Fixed Pattern Noise, FPN）

3.1.2.6.1 Test Pattern的用途和作用

1. **Sensor 功能验证/调试**

   在 **Sensor 开发 / 验证阶段**，工程师可以通过 Test Pattern 快速确认：

   - Sensor 是否正常输出图像数据
   - MIPI CSI-2 接口是否工作正常（数据通路是否 OK）
   - 图像的分辨率、色彩格式（如 RAW、RGB、YUV）是否正确
   - ISP 或下游模块是否能正确接收和处理数据流

   举个例子：

> 你刚给一个 Sensor 上电，不知道它有没有正常工作，也不想接真实的镜头/光源。
> **开启 Test Pattern 后，如果屏幕或调试工具上能看到彩色条纹/渐变图案，说明 Sensor 和数据通路基本是 OK 的！**

2. **ISP 调试与算法验证**

在 ISP（图像信号处理器）开发阶段，Test Pattern 可以用来：

- 验证 **ISP 各模块功能**（如 Demosaic、3A、Gamma、CSC、Sharpening 等）
- 测试 **不同输入图案下 ISP 的处理效果**
- 检查 **色彩还原、亮度、对比度、噪声抑制算法** 是否正常工作

  因为 Test Pattern 是已知的、可控的输入，所以非常适合用来做 **定量分析和算法调优**。

  比如：你输入一个 **纯色块（比如全红）**，看 ISP 输出是否也是纯红，可以验证 **白平衡 / 颜色校正模块是否异常**；输入 **渐变图案**，可以用来测试 **Gamma、动态范围、 tone mapping** 等功能。

3. **摄像头模组调试 & 校准**

在 **摄像头模组生产 / 校准阶段**，Test Pattern 可用于：

- 验证镜头模组 + Sensor + ISP 的数据通路
- 做自动对焦（AF）、自动曝光（AE）、自动白平衡（AWB）算法的模拟测试
- 在无真实光源的环境下进行基本功能测试

4. **故障排查 / 异常分析**

当摄像头输出图像 异常（如偏色、花屏、条纹、全黑/全白） 时，你可以：

- 先开启 Test Pattern，看是否仍然异常
  - 如果 **Test Pattern 输出正常** → 说明问题可能出在 **光学部分（如镜头脏污、无光）或真实场景输入**
  - 如果 **Test Pattern 输出也异常** → 说明问题在 **Sensor、MIPI 通路、ISP 或电源**

所以 Test Pattern 是一个非常重要的 **排查工具**，用来区分问题是出在 **光学链路** 还是 **电信号/数据处理链路**。

5. **工厂生产测试**

在摄像头 / Sensor 的 工厂量产测试环节，Test Pattern 可用于：

- 自动化测试 Sensor 输出是否正常
- 检查 坏点（Dead Pixel）、亮点（Hot Pixel）、固定噪声 等
- 验证 数据完整性、色彩准确性、动态范围等指标

3.1.2.6.2 常见的 Test Pattern 类型

Test Pattern 类型	描述	用途
Solid Color（单色）	整幅图像为纯色（如纯红 / 绿 / 蓝 / 黑 / 白）	检查色彩通道、坏点
Color Bars（彩色条纹）	类似电视测试图的彩色横条（红/绿/蓝/黑/白等组合）	验证色彩输出、ISP处理
Gradient（渐变）	从黑到白或红绿蓝的平滑过渡	测试动态范围、Gamma、色调映射
Checkerboard（棋盘格）	黑白相间的方格图案	测试锐利度、边缘处理、压缩算法
Moving Pattern（动态图案）	一些 Sensor 支持动态变化的测试图	测试时序、缓存、延迟
Random / Noise Pattern	模拟噪声图案	测试噪声抑制算法
Cross Hatch / Grid	交叉线网格	检查几何失真、对齐

```
#define SC200AI_REG_TEST_PATTERN        0x4501 //测试图案控制寄存器
#define SC200AI_TEST_PATTERN_BIT_MASK   BIT(3) //测试图案开启掩码
```

3.1.2.7 VTS

在 **CMOS 图像传感器（CIS）** 或 **摄像头模组** 的上下文中，**HTS** 和 **VTS** 是两个非常关键的时序参数。本节重点介绍 VTS。

VTS 是 **Vertical Total Time** 或 **Vertical Total Lines** 的简称。中文俗称：**帧总周期/帧总行数**。它表示 **传感器输出一帧完整图像（包括所有有效行 + 消隐行）所用的总行数（或总时间）**。通常是 **行数（lines）**，也可以换算为时间（比如 ms），取决于帧率与 VTS。

包含的内容：

1. **Active Lines（有效行数）**：即实际图像数据的行数，比如 1080（对于 1920x1080 分辨率中的高度）。
2. **VSYNC（场同步信号）**
3. **帧消隐区（Vertical Blank / 前廊/后廊）**：用于传感器切换帧、内部处理等。

VTS = 有效行数 + 垂直消隐行数

HTS 、VTS 与图像分辨率、帧率的关系：

1. 帧率（FPS）的计算

   帧率关键计算公式是：

$$FPS = PCLK / (HTS \times VTS)$$

其中：

- **PCLK**：Pixel Clock，即每个像素的时钟频率（单位通常是 MHz 或 Hz），表示每秒传输多少个像素时钟。

- **HTS**：每行总时钟周期数（决定一行花多长时间）
- **VTS**：一帧总行数（决定一帧花多少行）

所以，**HTS 和 VTS 共同决定了传感器的帧率！**如果你想 **提高帧率（FPS↑）**，你可以：

- 减少 **VTS（总行数，即减少垂直消隐）**
- 或提高 **PCLK（像素时钟）**

反之，如果你想降低帧率（比如低光下需要更长曝光），可以增加 VTS 或降低 PCLK。

HTS 与 VTS 的核心区别：

项目	HTS (Horizontal Total)	VTS (Vertical Total)
方向	水平方向（一行内）	垂直方向（一帧内）
含义	一行总的时钟周期数（包含有效像素+消隐）	一帧总行数（包含有效行+消隐）
包含内容	有效像素 + HSYNC + 水平消隐区	有效行数 + VSYNC + 垂直消隐区
影响	行频、数据传输时序	帧率、帧时间
与帧率关系	FPS = PCLK / (HTS × VTS)	增大 VTS 会降低帧率，减小则提高
单位	时钟周期数（或像素时钟数）	行数（lines）
典型配置位置	传感器寄存器（行时序）	传感器寄存器（帧时序）

```
#define SC200AI_REG_VTS_H       0x320e
#define SC200AI_REG_VTS_L       0x320f
```

3.1.2.8 Mirror / Flip

镜像（Mirror）翻转（Flip）控制寄存器。

在 **图像传感器（Sensor）** 的上下文中，**Mirror（镜像）** 和 **Flip（翻转）** 是两个用于控制图像 **输出方向或方位** 的功能，它们影响的是 **Sensor 输出的图像内容的左右或上下方向**，属于 **图像几何变换的范畴**。

1. **Mirror（镜像）** 是指将图像在 **水平方向（X 轴）上做左右对称翻转**，类似于人照镜子时看到的效果。就像你照镜子时，你的左侧在镜子里显示在右侧，这就是 **水平镜像（Mirror）**。

在 Sensor 中的作用：

- 通过设置 Sensor 寄存器中的 **Mirror 位（或 Horizontal Flip）**，可以让 Sensor 输出的图像在水平方向翻转。
- 常用于：
  - **Sensor 横向安装反了**，导致图像左右相反
  - 镜头安装方向导致需要水平翻转图像

2. **Flip（翻转）** 是指将图像在 **垂直方向（Y 轴）上做上下对称翻转**，也就是将图像**上下倒置**。就像你把一张照片 **旋转 180° 上下倒置**，或者从天花板往下看图像，就是 **垂直翻转（Flip）** 的效果。

在 Sensor 中的作用：

- 通过设置 Sensor 寄存器中的 **Flip 位（或 Vertical Flip）**，可以让 Sensor 输出的图像在垂直方向翻转。
- 常用于：
  - **Sensor 倒装（比如摄像头倒着装在设备中）**
  - 需要调整图像方向以匹配显示要求

Mirror 和 Flip 的组合效果

设置	效果	实际表现
无 Mirror / 无 Flip	正常输出	图像无翻转，和人眼看到的一致（默认方向）
Mirror（水平镜像）	左右翻转	图像左右对调，像照镜子一样
Flip（垂直翻转）	上下翻转	图像上下颠倒，像倒置的照片
Mirror + Flip	左右镜像 + 上下翻转 = 旋转 180°	图像整体看起来像是旋转了 180 度（上下左右全反）

在大多数 CMOS 图像传感器（如 **Sony、OmniVision、SmartSens、GalaxyCore、Onsemi 等**）中，**Mirror 和 Flip 功能通常是通过寄存器配置的**。

```
#define SC200AI_FLIP_MIRROR_REG     0x3221 // 镜像翻转控制寄存器

#define SC200AI_FETCH_MIRROR(VAL, ENABLE)   (ENABLE ? VAL | 0x06 : VAL & 0xf9)//设置镜像
#define SC200AI_FETCH_FLIP(VAL, ENABLE)     (ENABLE ? VAL | 0x60 : VAL & 0x9f)//设置翻转
```

3. v4l2-ctl命令设置镜像与翻转

```
// /dev/v4l-subdev2节点对应 Sensor 子设备节点。参考5.4 章节
v4l2-ctl --set-ctrl=horizontal_flip=1 -d /dev/v4l-subdev2  # 启用水平镜像
v4l2-ctl --set-ctrl=horizontal_flip=0 -d /dev/v4l-subdev2  # 禁用水平镜像

v4l2-ctl --set-ctrl=vertical_flip=1 -d /dev/v4l-subdev2    # 启用垂直翻转
v4l2-ctl --set-ctrl=vertical_flip=0  -d /dev/v4l-subdev2   # 禁用垂直翻转

v4l2-ctl --set-ctrl=horizontal_flip=1 --set-ctrl=vertical_flip=1 -d /dev/v4l-subdev2  # 同时启用镜像和翻转
v4l2-ctl --set-ctrl=horizontal_flip=0 --set-ctrl=vertical_flip=0  -d /dev/v4l-subdev2# 同时禁用镜像和翻转
```

3.2 驱动框架与说明

Camera Sensor 采用 I2C与主控进行交互，目前 Sensor的驱动按照 I2C 设备驱动方式实现，Sensor 的驱动同时采用 v4l2 subdev 的方式实现与host driver 之间的交互。

3.2.1 数据类型说明

3.2.1.1 struct i2c_driver

【说明】

定义 I2C 设备驱动信息

【定义】

```
struct i2c_driver {
    ...

    /* Standard driver model interfaces */
    int (*probe)(struct i2c_client *client, const struct i2c_device_id *id);
    int (*remove)(struct i2c_client *client);
    ...
    struct device_driver driver;
    const struct i2c_device_id *id_table;
    ...
};
```

【关键成员】

成员名称	数据类型	作用描述
`probe`	`int (*)(struct i2c_client *)`	设备探测函数，当驱动匹配到设备时调用
`remove`	`void (*)(struct i2c_client *)`	设备移除清理函数
`driver`	`struct device_driver`	**核心驱动结构,当of_match_table中的compatible域和dts文件的compatible域匹配时，.probe函数才会被调用**
`id_table`	`const struct i2c_device_id *`	**设备匹配表, 如果kernel没有使用of_match_table和dts注册设备进行进行匹配，则kernel使用该table进行匹配**

【示例】

```
#if IS_ENABLED(CONFIG_OF)
static const struct of_device_id sc200ai_of_match[] = {
    { .compatible = "smartsens,sc200ai" },
    {},
};
MODULE_DEVICE_TABLE(of, sc200ai_of_match);
#endif

static const struct i2c_device_id sc200ai_match_id[] = {
    { "smartsens,sc200ai", 0 },
    { },
```

```
};

static struct i2c_driver sc200ai_i2c_driver = {
    .driver = {
        .name = SC200AI_NAME,
        .pm = &sc200ai_pm_ops,
        .of_match_table = of_match_ptr(sc200ai_of_match),
    },
    .probe      = &sc200ai_probe,
    .remove     = &sc200ai_remove,
    .id_table   = sc200ai_match_id,
};

static int __init sensor_mod_init(void)
{
    return i2c_add_driver(&sc200ai_i2c_driver);
}

static void __exit sensor_mod_exit(void)
{
    i2c_del_driver(&sc200ai_i2c_driver);
}
```

3.2.1.2 struct v4l2_subdev_ops

【说明】

`struct v4l2_subdev_ops` 是 Linux V4L2 框架中 **子设备（Sub-device）操作集**的核心结构体，定义了各类硬件模块（如 Sensor、ISP、解码器）的功能接口

【定义】

```
struct v4l2_subdev_ops {
    const struct v4l2_subdev_core_ops   *core;
    const struct v4l2_subdev_video_ops  *video;
    const struct v4l2_subdev_pad_ops    *pad;
    ...
};
```

【关键成员】

成员类型	结构体	功能描述	适用硬件
核心操作	`const struct v4l2_subdev_core_ops *core`	基础控制（电源/中断/日志等）	所有子设备
视频流控制	`const struct v4l2_subdev_video_ops *video`	视频流参数（帧率/格式/HDR等）	Sensor/ISP/编码器
面板控制	`const struct v4l2_subdev_pad_ops *pad`	像素格式/分辨率/裁剪设置 (**关键**)	Sensor/ISP/MIPI-CSI接收器

【示例】

```
static const struct v4l2_subdev_ops sc200ai_subdev_ops = {
    .core   = &sc200ai_core_ops,
    .video  = &sc200ai_video_ops,
    .pad    = &sc200ai_pad_ops,
};
```

3.2.1.3 struct v4l2_subdev_core_ops

【说明】

V4L2 子设备驱动中用于实现核心控制功能的操作集。

【定义】

```
struct v4l2_subdev_core_ops {
    ...
    long (*ioctl)(struct v4l2_subdev *sd, unsigned int cmd, void *arg);
#ifdef CONFIG_COMPAT
    long (*compat_ioctl32)(struct v4l2_subdev *sd, unsigned int cmd,
                unsigned long arg);
#endif
    int (*s_power)(struct v4l2_subdev *sd, int on);
    ...
};
```

【关键成员】

成员函数	函数原型	功能描述	在 RK Sensor 驱动中的典型实现
`s_power`	`int (*)(struct v4l2_subdev *sd, int on)`	**控制电源状态**：开启/关闭设备供电	控制 PWDN/RESET 引脚的电平序列，管理 PMIC 供电时序（上电：`on=1`，掉电：`on=0`）
`compat_ioctl32`	`long (*compat_ioctl32)(struct v4l2_subdev *sd, unsigned int cmd, unsigned long arg)`		当 32bit 应用使用 64 位 kernel 时，实现私有功能，**为了解决从用户空间传递过来/传递到用户空间的数据**
`ioctl`	`int (*)(struct v4l2_subdev *sd, unsigned int cmd, void *arg)`	处理**私有自定义命令**（非标准V4L2命令）	实现 特有功能，响应系统 ioctl 调用

【示例】

```
static const struct v4l2_subdev_core_ops sc200ai_core_ops = {
    .s_power = sc200ai_s_power,
    .ioctl = sc200ai_ioctl,
#ifdef CONFIG_COMPAT
    .compat_ioctl32 = sc200ai_compat_ioctl32,
#endif
};c
```

目前使用了如下的私有ioctl命令，用来实现模组信息的查询和OTP信息的查询设置。

私有ioctl	描述
RKMODULE_GET_MODULE_INFO	获取模组信息
PREISP_CMD_SET_HDRAE_EXP	hdr 曝光设置
RKMODULE_SET_HDR_CFG	设置 hdr 模式，可实现 normal(线性) 和 hdr（高动态）模式切换，需要驱动适配 normal 和 hdr 2 组配置信息
RKMODULE_GET_HDR_CFG	获取当前hdr模式
RKMODULE_SET_CONVERSION_GAIN	设置线性模式的conversion gain，如imx347、os04a10 Sensor带有conversion gain 的功能，若当前 Sensor不支持conversion gain，可不实现。

3.2.1.4 struct v4l2_subdev_video_ops

【说明】

视频设备控制接口。

【定义】

```
struct v4l2_subdev_video_ops {
    ...
    int (*s_stream)(struct v4l2_subdev *sd, int enable);
    int (*g_frame_interval)(struct v4l2_subdev *sd,
                struct v4l2_subdev_frame_interval *interval);
    int (*s_frame_interval)(struct v4l2_subdev *sd,
                struct v4l2_subdev_frame_interval *interval);
    ...

};
```

【关键成员】

成员名称	描述
`.g_frame_interval`	VIDIOC_SUBDEV_G_FRAME_INTERVAL ioctl 的回调函数
`.s_stream`	通知驱动视频流启动或停止的关键接口
`.s_frame_interval`	动态修改 **Sensor 的输出帧率**（帧间隔），是调整帧率的核心回调函数

【示例】

```
static const struct v4l2_subdev_video_ops sc200ai_video_ops = {
    .s_stream = sc200ai_s_stream,
    .g_frame_interval = sc200ai_g_frame_interval,
    .s_frame_interval = sc200ai_s_frame_interval,
};
```

3.2.1.5 struct v4l2_subdev_pad_ops

【说明】

此结构体定义子设备在 **像素格式协商、裁剪、缩放和帧布局** 等操作的回调函数集合; 在 RK 等平台的 Sensor 驱动中，它是配置图像处理流水线（Sensor → ISP → 内存）的关键接口。

【定义】

```
struct v4l2_subdev_pad_ops {
    ...
    int (*enum_mbus_code)(struct v4l2_subdev *sd,
                  struct v4l2_subdev_pad_config *cfg,
                  struct v4l2_subdev_mbus_code_enum *code);
```

```
    int (*enum_frame_size)(struct v4l2_subdev *sd,
                   struct v4l2_subdev_pad_config *cfg,
                   struct v4l2_subdev_frame_size_enum *fse);
    int (*enum_frame_interval)(struct v4l2_subdev *sd,
                   struct v4l2_subdev_pad_config *cfg,
                   struct v4l2_subdev_frame_interval_enum *fie);
    int (*get_fmt)(struct v4l2_subdev *sd,
                struct v4l2_subdev_pad_config *cfg,
                struct v4l2_subdev_format *format);
    int (*set_fmt)(struct v4l2_subdev *sd,
                struct v4l2_subdev_pad_config *cfg,
                struct v4l2_subdev_format *format);
    int (*get_selection)(struct v4l2_subdev *sd,
                  struct v4l2_subdev_pad_config *cfg,
                  struct v4l2_subdev_selection *sel);
    int (*set_selection)(struct v4l2_subdev *sd,
                  struct v4l2_subdev_pad_config *cfg,
                  struct v4l2_subdev_selection *sel);
    int (*get_edid)(struct v4l2_subdev *sd, struct v4l2_edid *edid);
    int (*set_edid)(struct v4l2_subdev *sd, struct v4l2_edid *edid);
    int (*dv_timings_cap)(struct v4l2_subdev *sd,
                   struct v4l2_dv_timings_cap *cap);
    int (*enum_dv_timings)(struct v4l2_subdev *sd,
                   struct v4l2_enum_dv_timings *timings);
    ...

    int (*get_mbus_config)(struct v4l2_subdev *sd, unsigned int pad,
                   struct v4l2_mbus_config *config);
    int (*set_mbus_config)(struct v4l2_subdev *sd, unsigned int pad,
                   struct v4l2_mbus_config *config);
};
```

【关键成员】

成员函数	功能描述	Rockchip 平台典型应用
`enum_mbus_code`	枚举支持的媒体总线格式（像素格式）	报告 Sensor 支持的 RAW/YUV 格式（如 `MEDIA_BUS_FMT_SRGGB10_1X10`）
`get_fmt`	获取当前设置的格式（分辨率 + 像素格式）	查询 Sensor 当前输出配置（如 `1920x1080@SRGGB10`）
`set_fmt`	**设置格式**（分辨率 + 像素格式），驱动中**最重要**的函数	配置 Sensor 输出分辨率（如 1920x1080）和格式（如 NV12），影响 MIPI 数据流
`enum_frame_size`	枚举指定像素格式下支持的分辨率	列出特定格式（如 UYVY）可用的分辨率（如 640x480, 1280x720）
`enum_frame_interval`	枚举指定格式和分辨率下支持的帧率	报告当前模式下支持的分辨率、帧率等
`get_selection`	获取裁剪/缩放区域（读/写区域，输出尺寸）	实现数码变焦，或设置低功耗模式下的小分辨率区域
`set_selection`	设置裁剪/缩放区域	在预览模式中使用 1080p 区域
`get_edid` / **`set_edid`**	处理 EDID（显示器扩展标识数据）	HDMI 接收芯片专用，Sensor 驱动通常忽略
`dv_timings_cap`	获取支持的数字视频时序标准	HDMITX/RX 芯片专用
`enum_dv_timings`	枚举支持的数字视频时序	HDMITX/RX 芯片专用
`get_mbus_config`	获取媒体总线配置（如 MIPI D-PHY 参数）	报告： • 通道数：`V4L2_MBUS_CSI2_4LANE` • 时钟模式：`V4L2_MBUS_CSI2_CONTINUOUS_CLOCK`
`set_mbus_config`	设置媒体总线配置	动态调整 MIPI 参数（如切换 2-lane/4-lane 模式）

【示例】

```
static const struct v4l2_subdev_pad_ops sc200ai_pad_ops = {
    .enum_mbus_code = sc200ai_enum_mbus_code,
    .enum_frame_size = sc200ai_enum_frame_sizes,
    .enum_frame_interval = sc200ai_enum_frame_interval,
    .get_fmt = sc200ai_get_fmt,
    .set_fmt = sc200ai_set_fmt,
    .get_mbus_config = sc200ai_g_mbus_config,
};
```

3.2.1.6 struct v4l2_ctrl_ops

【说明】

`struct v4l2_ctrl_ops` 是 **V4L2 控制框架的核心操作集**，用于实现传感器参数（如曝光、增益、白平衡等）的动态控制。在 Rockchip 平台 Sensor 驱动开发中，它提供了标准化接口来处理应用层对传感器参数的配置请求，是连接用户空间控制与硬件寄存器操作的关键枢纽。

【定义】

```
struct v4l2_ctrl_ops {
    ...
    int (*s_ctrl)(struct v4l2_ctrl *ctrl);
};
```

【关键成员】

成员函数	功能描述	Rockchip 典型实现案例
`s_ctrl`	**设置控制值**（最重要函数）	将用户空间设置的值写入硬件寄存器（如设置曝光时间/增益）

【示例】

```
static int sc200ai_set_ctrl(struct v4l2_ctrl *ctrl)
{
    struct sc200ai *sc200ai = container_of(ctrl->handler,
                           struct sc200ai, ctrl_handler);
    struct i2c_client *client = sc200ai->client;

    ...

    if (!pm_runtime_get_if_in_use(&client->dev))
        return 0;

    switch (ctrl->id) {
    case V4L2_CID_EXPOSURE:
        dev_dbg(&client->dev, "set exposure value 0x%x\n", ctrl->val);
        ...
        break;
    case V4L2_CID_ANALOGUE_GAIN:
        dev_dbg(&client->dev, "set gain value 0x%x\n", ctrl->val);
        ...
        break;
    case V4L2_CID_VBLANK:
        dev_dbg(&client->dev, "set blank value 0x%x\n", ctrl->val);
        ...
        break;
    case V4L2_CID_TEST_PATTERN:
        ...
        break;
    case V4L2_CID_HFLIP:
        ...
```

```
        break;
    case V4L2_CID_VFLIP:
        ...
        break;
    default:
        dev_warn(&client->dev, "%s Unhandled id:0x%x, val:0x%x\n",
             __func__, ctrl->id, ctrl->val);
        break;
    }

    pm_runtime_put(&client->dev);

    return ret;
}

static const struct v4l2_ctrl_ops sc200ai_ctrl_ops = {
    .s_ctrl = sc200ai_set_ctrl,
};
```

3.2.2 API介绍

3.2.2.1 xxxx_set_fmt

【描述】

设置 Sensor 输出格式。这里指的是一个 Sensor 驱动内支持多种分辨率。通过 set_fmt 下发分辨率，Sensor 驱动内遍历去获得最符合的分辨率。
HDR 和线性模式的切换不通过这个函数配置，通过 ioctl 实现私有命令，实现切换。需要注意的是 HDR 和线性模式的分辨率要一致才能实现切换。

【语法】

```
static int xxxx_set_fmt(struct v4l2_subdev *sd,
                        struct v4l2_subdev_pad_config *cfg,
                        struct v4l2_subdev_format *fmt)
```

【参数】

参数名称	描述	输入输出
*sd	v4l2 subdev结构体指针	输入
*cfg	subdev pad information结构体指针	输入
*fmt	Pad-level media bus format结构体指针	输入

【返回值】

返回值	描述
0	成功
非0	失败

【注意事项】

kernel 6.1版本中`struct v4l2_subdev_pad_config *cfg`改为了`struct v4l2_subdev_state *sd_state`

3.2.2.2 xxxx_get_fmt

【描述】

获取 Sensor 输出格式，获取到的是当前使用的配置，如前面描述的 struct xxx_mode 里面可能配置多个不同分辨率的配置，驱动初始化的时候会选中其中一个，或者 set_fmt 切换后，可通过 get_fmt 来确认当前的 format。
对于 RK3588 以前的芯片，这边还会通过 fmt->reserved[0] 这个保留参数上传当前设备使用的 vc 通道信息，也就是可以通过这个参数来指定设备的vc通道，否则按默认值。
对于RK3588及后面的芯片，则通过ioctl实现RKMODULE_GET_CHANNEL_INFO，来获取通道信息。

【语法】

```
static int xxxx_get_fmt(struct v4l2_subdev *sd,
                        struct v4l2_subdev_pad_config *cfg,
                        struct v4l2_subdev_format *fmt)
```

【参数】

参数名称	描述	输入输出
*sd	v4l2 subdev结构体指针	输入
*cfg	subdev pad information结构体指针	输入
*fmt	Pad-level media bus format结构体指针	输出

【返回值】

返回值	描述
0	成功
非0	失败

【注意事项】

kernel 6.1版本中`struct v4l2_subdev_pad_config *cfg`改为了`struct v4l2_subdev_state *sd_state`

3.2.2.3 xxxx_enum_mbus_code

【描述】

枚举 Sensor 输出 bus format，驱动中会根据 struct xxx_mode 结构定义一个静态结构体变量，里面填充 Sensor 驱动支持的各个分辨率，这个函数可以根据结构体变量里面填充的 bus format，返回枚举值。

【语法】

```
static int xxxx_enum_m bus_code(struct v4l2_subdev *sd,
                                struct v4l2_subdev_pad_config *cfg,
                                struct v4l2_subdev_mbus_code_enum *code)
```

【参数】

参数名称	描述	输入输出
*sd	v4l2 subdev结构体指针	输入
*cfg	subdev pad information结构体指针	输入
*code	media bus format enumeration结构体指针	输出c

【返回值】

参数名称	描述	输入输出
*sd	v4l2 subdev结构体指针	输入
*cfg	subdev pad information结构体指针	输入
*code	media bus format enumeration结构体指针	输出

【注意事项】

kernel 6.1版本中`struct v4l2_subdev_pad_config *cfg`改为了`struct v4l2_subdev_state *sd_state`。

3.2.2.4 xxxx_enum_frame_sizes

【描述】

枚举 Sensor 输出大小。驱动中会根据 struct xxx_mode 结构定义一个静态结构体变量，里面填充 Sensor 驱动支持的各个分辨率，这个函数可以根据结构体变量里面填充的分辨率，返回枚举值。

【语法】

```
static int xxxx_enum_frame_sizes(struct v4l2_subdev *sd,
                                 struct v4l2_subdev_pad_config *cfg,
                                 struct v4l2_subdev_frame_size_enum *fse)
```

【参数】

参数名称	描述	输入输出
*sd	v4l2 subdev结构体指针	输入
*cfg	subdev pad information结构体指针	输入
*fse	media bus frame size结构体指针	输出

【返回值】

返回值	描述
0	成功
非0	失败

【注意事项】

kernel 6.1 版本中`struct v4l2_subdev_pad_config *cfg`改为了`struct v4l2_subdev_state *sd_state`。

3.2.2.5 xxxx_g_frame_interval

【描述】

获取 Sensor 输出帧间隔。驱动中会根据 struct xxx_mode 结构定义一个静态结构体变量，里面的配置的 帧率是最高帧率，驱动通过增加 vblank 的值来降低帧率，如果需要获取实际帧率，这边返回给应用的值，需要将默认的vblank和当前的 vblank值进行换算，以获取最新的帧间隔，上传给应用。

【语法】

```
static int xxxx_g_frame_interval(struct v4l2_subdev *sd,
                                 struct v4l2_subdev_frame_interval *fi)
```

【参数】

参数名称	描述	输入输出
*sd	v4l2 subdev结构体指针	输入
*fi	pad-level frame rate结构体指针	输出

【返回值】

返回值	描述
0	成功
非0	失败

3.2.2.6 xxxx_s_stream

【描述】

设置stream输入输出。

函数一般根据传入的参数 on 实现 start_stream/stop_stream 函数。start_stream 里面实现初始化寄存器数组的配置，初始化曝光参数配置，开启数据流。 stop_stream里面实现 stream off 的寄存器配置。

【语法】

```
static int xxxx_s_stream(struct v4l2_subdev *sd, int on)
```

【参数】

参数名称	描述	输入输出
*sd	v4l2 subdev结构体指针	输入
on	1: 启动stream输出; 0: 停止stream输出	输入

【返回值】

返回值	描述
0	成功
非0	失败

3.2.2.7 xxxx_runtime_resume

【描述】

Sensor上电时的回调函数。

函数里面主要做 Sensor上电操作，将上电操作放在这个函数，方便 Sensor 驱动其他地方或者上层驱动调用pm_runtime_get_sync来上电。。

【语法】

```
static int xxxx_runtime_resume(struct device *dev)
```

【参数】

参数名称	描述	输入输出
*dev	device结构体指针	输入

【返回值】

返回值	描述
0	成功
非0	失败

3.2.2.8 xxxx_runtime_suspend

【描述】

Sensor 下电时的回调函数。

函数里面做 Sensor 的下电操作，方便 Sensor 驱动其他地方或者上层驱动调用 pm_runtime_put 来下电。

【语法】

```
static int xxxx_runtime_suspend(struct device *dev)
```

【参数】

参数名称	描述	输入输出
*dev	device结构体指针	输入

【返回值】

返回值	描述
0	成功
非0	失败

3.2.2.9 xxxx_set_ctrl

【描述】

设置各个 v4l2 control 的值。

注册 v4l2 control 命令时，调用这个回调函数，应用设置 control 参数下来时，通过这个回调函数来配置 Sensor。

需要实现的 v4l2 control 命令可以参考 3.2.1.6 参考struct v4l2_ctrl_ops 说明实现 。

【语法】

```
static int xxxx_set_ctrl(struct v4l2_ctrl *ctrl)
```

【参数】

参数名称	描述	输入输出
*ctrl	v4l2_ctrl结构体指针	输入

【返回值】

返回值	描述
0	成功
非0	失败

3.2.2.10 xxx_enum_frame_interval

【描述】

枚举 Sensor 支持的帧间隔参数。

驱动中会根据 struct xxx_mode 结构定义一个静态结构体变量，可以将里面支持的几种分辨率下的帧间 隔枚举上去。这边一般用途不大，对应帧率调整还是通过调整vblank。

【语法】

```
static int xxxx_enum_frame_interval(struct v4l2_subdev *sd,
                                    struct v4l2_subdev_pad_config *cfg,
                                    struct v4l2_subdev_frame_interval_enum *fie)
```

【参数】

参数名称	描述	输入输出
*sd	子设备实例	输入
*cfg	pad配置参数	输入
*fie	帧间隔参数	输出

【返回值】

返回值	描述
0	成功
非0	失败

【注意事项】

kernel 6.1 版本中`struct v4l2_subdev_pad_config *cfg`改为了`struct v4l2_subdev_state *sd_state`。

3.2.2.11 xxxx_g_mbus_config

【描述】

获取支持的总线配置，总线类型存在并 /MIPI/LVDS 等几种类型，并口里面存在 BT601/BT656/BT1120，MIPI 存在 DPHY/CPHY 协议，控制器通过这个接口获取当前 Sensor 使用的总线参 数，来确认控制器的采集方式。

比如使用 MIPI 时， 当 Sensor 支持多种 MIPI 传输模式时，可以根据 Sensor 当前使用的 MIPI 模式上传参数。

【语法】

```
static int xxxx_g_m bus_config(struct v4l2_subdev *sd,
                               struct v4l2_m bus_config *config)
```

【参数】

参数名称	描述	输入输出
*config	总线配置参数	输出c

【返回值】

参数名称	描述	输入输出
*config	总线配置参数	输出

3.2.2.12 xxxx_get_selection

【描述】

配置裁剪参数，ISP 输入的宽度要求16对齐，高度8对齐，对于 Sensor 输出的分辨率不符合对齐或 Sensor 输出分辨率不是标准分辨率，可实现这个函数对输入 ISP 的分辨率做裁剪。

【语法】

```
static int xxxx_get_selection(struct v4l2_subdev *sd,
                              struct v4l2_subdev_pad_config *cfg,
                              struct v4l2_subdev_selection *sel)
```

【参数】

参数名称	描述	输入输出
*sd	子设备实例	输入
*cfg	pad配置参数	输入
*sel	裁剪参数	输出

【返回值】

返回值	描述
0	成功
非0	失败

【注意事项】

kernel 6.1 版本中`struct v4l2_subdev_pad_config *cfg`改为了`struct v4l2_subdev_state *sd_state`。

3.2.2.13 xxxx_ioctl

【描述】

ioctl 私有命令实现。

【语法】

```
static long xxxx_ioctl(struct v4l2_subdev *sd, unsigned int cmd, void *arg)
```

【参数】

参数名称	描述	输入输出
*sd	子设备实例	输入
cmd	私有命名值	输入
* arg	传递的参数	输入/输出

【返回值】

返回值	描述
0	成功
非0	

3.3 驱动移植步骤

3.3.1 实现标准I2C子设备驱动部分

1. 根据struct i2c_driver说明实现以下成员:

   struct driver.name

   struct driver.pm

   struct driver. of_match_table probe函数

   remove函数。（注意 kenrel 5.10 与 6.1 版本返回值的区别）

2. probe函数实现细节描述:

- CIS 设备资源的获取,主要是解析 DTS 文件中定义资源。
  - RK私有资源定义,命名方式如下 `rockchip,camera-module-xxx`, 该部分资源会由驱动上传给用户态的 camera_engine 来决定 IQ 效果参数的匹配;
  - CIS设备资源定义, RK相关参考驱动一般包含以下几项:

成员名称	描述
CIS设备工作参考时钟	采用外部独立晶振方案无需获取, RK 参考设计一般采用 AP 输出时钟 ， 该方案需要获取 ， 一般名称为 xvclk。
CIS设备控制GPIO	例如： Resst引脚, Powerdown 引脚
CIS设备控制电源	根据实际硬件设计,获取匹配的软件电源控制资源, 例如 gpio, regulator

  - CIS 设备 ID号检查, 通过以上步骤获取必要资源后,建议驱动读取设备 ID 号以便检查硬件的准确性, 当然 该步骤非必要步骤。
  - CIS v4l2 设备以及 media 实体的初始化;

    v4l2子设备：v4l2_i2c_subdev_init ， RK CIS 驱动要求 subdev 拥有自己的设备节点供用户态 rk_aiq 访问 ，通过该设备节点实现曝光控制；

    media实体： media_entity_init

3.3.2 实现v4l2子设备驱动

主要实现以下3个成员：

```
struct v4l2_subdev_core_ops
struct v4l2_subdev_video_ops
struct v4l2_subdev_pad_ops
```

1. 参考 `struct v4l2_subdev_core_ops` 说明实现其回调函数 ， 主要实现以下回调：

- **`.s_power`**：

  s_power 实现 Sensor 的上下电操作 ， 对于一些寄存器数组较长的 Sensor ， 可以归纳出公共部分的寄存器 ， 在上电后执行 ， s_stream 只配置不同分辨率配置必须的寄存器 ， 加快切换速度。

- .compat_ioctl32:
- .ioctl:

  ioctl 主要实现的 RK 私有控制命令 ， 涉及:

成员名称	描述
RKMODULE_GET_MODULE_INFO	DTS 文件定义的模组信息(模组名称等)，通过该命令上 传给 camera_engine，用于设备匹配
RKMODULE_AWB_CFG	模组 OTP信息使能情况下，camera_engine 通过该命 令传递典型模组 AWB 标定值，CIS 驱动负责与当前模组 AWB 标定值比较后，生成 R/B Gain 值设置到 CIS MWB 模块中；
RKMODULE_LSC_CFG	模组 OTP 信息使能情况下，camera_engine 通过该命 令控制 LSC标定值生效使能；
PREISP_CMD_SET_HDRAE_EXP	HDR 曝光设置
RKMODULE_SET_HDR_CFG	设置 HDR 模式，可实现normal和hdr切换，需要驱动 适配 HDR 和normal 2组配置信息
RKMODULE_GET_HDR_CFG	获取当前 HDR 模式
RKMODULE_SET_CONVERSION_GAIN	设置线性模式的 conversion gain，如imx347、 os04a10 Sensor 带有conversion gain 的功能，高转换 的 conversion gain 可以在低照度下获得更好的信噪 比，如 Sensor不支持 conversion gain，可不实现
RKMODULE_SET_QUICK_STREAM	配置 Sensor的 stream on/off 寄存器，用于异常复位; 支持多摄同步，用于统一开启数据流，详 细参考多摄同步机制; 快速启动功能；AOV 功能；
RKMODULE_GET_CHANNEL_INFO	获取通道信息，默认 vicap 的 id0~id3 对应 vc0~vc3，如果有特殊需求可通过这个命令配置通道信 息，如 SPD/EBD 数据采集。RK3588 以前芯片仍通过 get_fmt 获取通道信息。
RKMODULE_GET_SYNC_MODE、RKMODULE_SET_SYNC_MODE	同步模式获取及配置，详细参考多摄同步

2. 参考 `struct v4l2_subdev_core_ops` 说明实现其回调函数 。

   主要实现以下回调:

成员名称	描述
.s_stream	开关数据流的函数，对于 MIPI CLK 是 continuous 的模式，必须在这个回调函数内开启数据流，若提前开数据流，会识别不到 MIPI LP 状态
.g_frame_interval	获取帧间隔参数（帧率）
.g_m bus_config	获取总线配置，对于MIPI接口，sensor驱动内若支持不同lane数配置或者 支持HDR,通过这个接口返回当前sensor工作模式下的MIPI配置

3. 参考 `struct v4l2_subdev_pad_ops` 说明实现其回调函数 。

   主要实现以下回调函数：

成员名称	描述
.enum_m bus_code	枚举当前 CIS 驱动支持数据格式，实现参考xxxx_enum_mbus_code
.enum_frame_size	枚举当前 CIS 驱动支持分辨率，实现参考xxxx_enum_frame_sizes
.get_fmt	RKISP driver 通过该回调获取CIS输出的数据格式，务必实现；针对 Bayer raw Sensor、SoC yuv Sensor、 BW raw Sensor 输出的数据类型 定义参考 MEDIA_BUS_FMT 列表针对 field 输出方式的支持，参考 struct v4l2_m bus_framefmt 定义； 实现参考 xxxx_get_fmt
.set_fmt	设置CIS驱动输出数据格式以及分辨率，务必实现，实现参考 xxxx_set_fmt
.enum_frame_interval	枚举sensor支持的帧间隔，包含分辨率。实现参考 xxx_enum_frame_interval
.get_selection	配置裁剪参数，ISP 输入的宽度要求16对齐，高度8对齐。 实现参考 xxxx_get_selection

4. 参考 `struct v4l2_ctrl_ops` 说明实现。

   主要实现以下回调函数：

成员名称	描述
.s_ctrl	RKISP driver、camera_engine 通过设置不同的命令来实现 CIS 曝光控制；

其中以下 ID 属于信息获取类，这部分实现按照 `standard integer menu controls` 方式实现：

成员名称	描述
V4L2_CID_LINK_FREQ	参考CIS驱动V4L2-controls列表中标准定义， 目前RKISP driver根据 该命令获取MIPI总线频率；
V4L2_CID_PIXEL_RATE	针对MIPI总线： pixel_rate = link_freq * 2 * nr_of_lanes / bits_per_sample
V4L2_CID_HBLANK	参考CIS驱动V4L2-controls列表中标准定义
V4L2_CID_VBLANK	参考CIS驱动V4L2-controls列表中标准定义

RK camera_engine 会通过以上命令获取必要信息来计算曝光，其中涉及的公式如下：

公式
line_time = HTS / PIXEL_RATE;
PIXEL_RATE = HTS * VTS * FPS
HTS = sensor_width_out + HBLANK;
VTS = sensor_height_out + VBLANK;

其中以下 ID 属于控制类，RK camera_engine 通过该类命令控制 CIS：

成员名称	描述
V4L2_CID_VBLANK	调整VBLANK，进而调整 frame rate、 Exposure time max；
V4L2_CID_EXPOSURE	设置曝光时间，单位： 曝光行数
V4L2_CID_ANALOGUE_GAIN	设置曝光增益，实际为 total gain = analog gain*digital gain; 单 位：增益寄存器值
V4L2_CID_HFLIP	设置水平镜像，ISP 不带镜像功能 ，如果需要镜像，使用 Sensor 的镜像机制
V4L2_CID_VFLIP	设置垂直镜像，ISP 不带镜像功能 ，如果需要镜像，使用 Sensor 的镜像机制
V4L2_CID_TEST_PATTERN	实现 test pattern，非必须，可用于调试，根据需要实现

5. HDR 驱动实现注意事项：

- 线性模式 exposure、gain 通过标准 v4l2 命令实现， 当切换成 HDR 模式时，通过ioctl命令 PREISP_CMD_SET_HDRAE_EXP 传递参数，标准 v4l2 命令中的 V4L2_CID_EXPOSURE、V4L2_CID_ANALOGUE_GAIN 不使用。PREISP_CMD_SET_HDRAE_EXP 实现时需要能够缓存参数，在 start_stream 写完初始化数组后，需要将初始曝光重新写进 Sensor，覆盖掉初始化数组里面的默认值，这样数据流输出的第一帧亮度才能和曝光参数匹配，才能尽快完成 AE 收敛。需要注意的是初始化数组里 面不能配置开启数据流的寄存器，需要在 start_stream 返回前配置，同样是为了保证输出的第一帧亮度 和初始化曝光参数匹配。
- Sensor 与控制器的 HDR/LINEAR 交互，通过RKMODULE_SET_HDR_CFG/RKMODULE_GET_HDR_CFG，HDR Sensor 必须实现，才能正常进行模式切换。

3.4 Makefile修改

修改 `kernel/drivers/media/i2c/Makefile` 文件，添加新增 Sensor 的编译。

以 SC200AI为例：

```
......
obj-$(CONFIG_VIDEO_SC200AI) += sc200ai.o
......
```

注意：

配置按照相同厂家及 Sensor 命名字母顺序排列添加。

3.5 Kconfig修改

修改 `kernel/drivers/media/i2c/Kconfig` 文件，添加新增 Sensor 的配置与描述

以 SC200AI为例：

```
config VIDEO_SC200AI
    tristate "SmartSens SC200AI sensor support"
    depends on I2C && VIDEO_V4L2
    select MEDIA_CONTROLLER
    select VIDEO_V4L2_SUBDEV_API
    select V4L2_FWNODE
    help
      This is a Video4Linux2 sensor driver for the SmartSens
      SC200AI camera.
```

注意：

配置按照相同厂家即 Sensor 命名字母顺序排列添加。

4. 快速启动

4.1 概念解释

Sensor 快启模式 是指 **图像传感器（如 CMOS Sensor）支持的一种快速上电、初始化并进入工作状态的模式**，目的是**让 Sensor 在尽可能短的时间内完成启动、初始化和输出有效图像数据，从而满足系统对快速启动、低延时、实时性的要求。**

在许多嵌入式系统、消费电子、汽车电子或工业视觉应用中，**系统上电后要求摄像头能够"尽快"输出图像**，比如

- 开机即拍 / 快速抓拍
- 系统启动时间要求严苛（比如 < 100ms / 300ms 内出图）
- 省电模式下唤醒摄像头并快速出图
- 多摄像头系统中需要快速切换或同步启动
- 视频通话 / 监控等实时性要求高的场景

传统的 Sensor 启动流程可能需要较长的时间（比如几百毫秒甚至超过 1 秒），包括：

1. 上电
2. 内部 PLL / 时钟稳定
3. 寄存器初始化
4. 模拟电路稳定
5. 行场时序建立
6. 输出第一帧有效图像

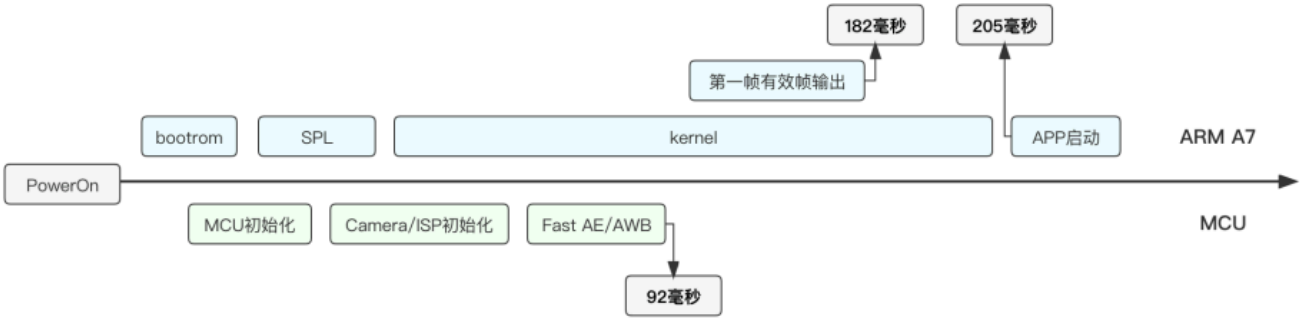
而快启模式，通过优化传感器内部的启动时序、跳过某些非必要的初始化步骤、或者提前配置好关键寄存器，从而让 Sensor **更快进入可输出图像的状态**。

不同启动模式，典型启动时间对比

启动模式	典型启动到出图时间	适用场景
普通启动（Normal Boot）	300ms ~ 1000ms	普通应用，对启动时间无严格要求
Fast Boot（快启模式）	10ms ~ 200ms	快速抓拍、实时预览、低延时系统
从 Standby 唤醒（Low-power Wakeup）	10ms ~ 50ms	省电模式下的快速恢复（配合 Fast Boot 更佳）

RK RV1103、RV1103B\RV1106B、RV1126B等主控平台内置一个高性能的 MCU RISCV 小核，将 ISP AE AWB 的部分功能移植到 MCU上，令其协助 ARM 初始化 Camera 和 ISP，实现图像效果快速收敛。从上电至 LINUX 内核系统启动需要一定的时间，在此阶段，MCU并行实现AE自动曝光收敛及日夜模式判断。内核 ISP 驱动注册的时候，Sensor 使用 MCU计算的收敛曝光量进行曝光出图，相关图像效果参数从 MCU 获取，进而达到快速抓取有效第一帧的目的，同时图像效果基本是稳定收敛的。

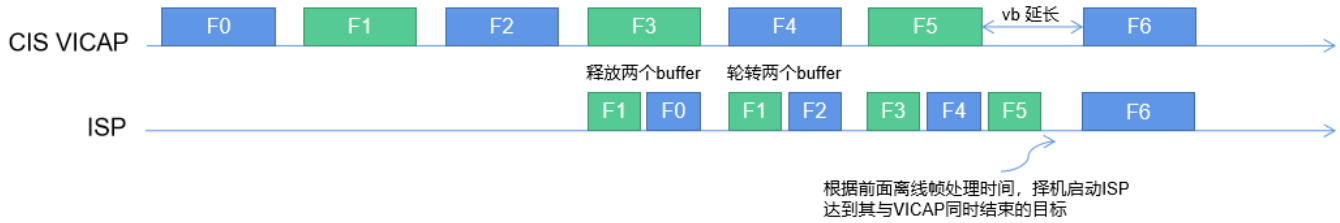
快起基本流程如图所示：



MCU辅助进行图像效果快速收敛的流程如下图：



ARM阶段出流具体流程如下：



4.2 驱动相关

4.2.1 内核驱动

1. 包含以下两个头文件：

```
#include "../platform/rockchip/isp/rkisp_tb_helper.h"
#include "cam-tb-setup.h"
```

2. Sensor 设备结构体中添加 is_thunderboot 变量，用来标识是否是快启模式。

```
struct sc200ai {
    ......
    bool            is_thunderboot;
    ......
};
```

probe 接口中判断是否配置了快启相关配置：CONFIG_VIDEO_ROCKCHIP_THUNDER_BOOT_ISP

若kernel 中配置了该配置项，is_thunderboot 为 true。

```
sc200ai->is_thunderboot = IS_ENABLED(CONFIG_VIDEO_ROCKCHIP_THUNDER_BOOT_ISP);
```

3. 起流、停流接口

```
static int __sc200ai_start_stream(struct sc200ai *sc200ai)
{
    int ret = 0;
    ......
    if (!sc200ai->is_thunderboot) { //非快启，写入完整寄存器配置
        ret = sc200ai_write_array(sc200ai->client, sc200ai->cur_mode->reg_list);
        if (ret)
            return ret;
        ......
    }

    if (sc200ai->sync_mode == NO_SYNC_MODE)//否则，配置起流
        ret |= sc200ai_write_reg(sc200ai->client, SC200AI_REG_CTRL_MODE,
                     SC200AI_REG_VALUE_08BIT, SC200AI_MODE_STREAMING);
    return ret;
}

static int __sc200ai_stop_stream(struct sc200ai *sc200ai)
{
    int ret = 0;
    sc200ai->has_init_exp = false;

    if (sc200ai->is_thunderboot) {
        sc200ai->is_first_streamoff = true;
        pm_runtime_put(&sc200ai->client->dev);
```

```
    }
    ......

    ret |= sc200ai_write_reg(sc200ai->client, SC200AI_REG_CTRL_MODE,
                SC200AI_REG_VALUE_08BIT, SC200AI_MODE_SW_STANDBY);
    return ret;
}
```

4.2.2 MCU驱动

参考摄像头驱动移植

4.3 MCU 快启配置指南

4.3.1 驱动框架与说明

以下章节均以思特威 SC450AI Sensor 驱动开发为例，说明 RK 平台 MCU Sensor 驱动开发的流程。

4.3.1.1 数据类型说明

4.3.1.1.1 struct rk_camera_ops

【说明】

struct rk_camera_ops定义了相机设备的操作接口。

【定义】

```
struct rk_camera_ops {
    ret_err_t (*init)(struct rk_camera_device *dev);  // 初始化相机设备
    ret_err_t (*open)(struct rk_camera_device *dev, rt_uint16_t oflag);  // 打开相机设备
    ret_err_t (*close)(struct rk_camera_device *dev);  // 关闭相机设备
    ret_err_t (*control)(struct rk_camera_device *dev, int cmd, void *args);  // 控制相机设备
};
```

【成员】

成员名称	描述
`init`	初始化设备
`open`	打开设备
`close`	关闭设备
`control`	控制设备

【示例】

```
struct rk_camera_ops rk_sc450ai_ops =
{
    rk_sc450ai_init,
    rk_sc450ai_open,
    NULL,
    rk_sc450ai_control
};
```

4.3.1.1.2 struct xxx_mode

【说明】

Sensor能支持各个模式的信息，该结构体可根据需求增加变量。

【定义】

```
struct xxxx_mode
{
    struct v4l2_fract max_fps;
    uint16_t bus_fmt;
    uint16_t width;
    uint16_t height;
    uint16_t hts_def;
    uint16_t vts_def;
    uint16_t exp_def;
    uint32_t xvclk;
    uint32_t link_freq;
#ifndef REG_LIST_CONVERSION_DEBUG
    const uint8_t *reg_list;
#else
    const struct sc450ai_sensor_reg *reg_list;
#endif
    const int reg_list_size;
    uint8_t hdr_mode;
} __attribute__((__packed__));
```

【成员】

成员名称	描述
.max_fps	图像帧率，denominator/numerator = fps。
.bus_fmt	Sensor输出格式。
.width	有效图像宽度。需要和sensor当前配置的width输出一致。
.height	有效图像高度。需要和sensor当前配置的height输出一致。
.hts_def	默认HTS，为有效图像宽度 + HBLANK。
.vts_def	默认VTS，为有效图像高度 + VBLANK。
.exp_def	默认曝光时间。
.xvclk	Sensor 外部时钟频率，即MCLK
.link_freq	MIPI_DPHY的采样频率（单位MHz），为MIPI速率的一半
*reg_list	寄存器列表
.reg_list_size	寄存器数组大小
.hdr_mode	Sensor工作模式，支持线性模式，两帧合成HDR,三帧合成HDR

【示例】

```
static const struct sc450ai_mode supported_modes_4lane[] =
{
    {
        .bus_fmt = MEDIA_BUS_FMT_SBGGR10_1X10,
        .width = 1344,
        .height = 760,
        .max_fps = {
            .numerator = 10000,
            .denominator = 1200000,
        },
        .exp_def = 0x80,
        .hts_def = 0x03a8,
        .vts_def = 0x030c,
        .xvclk = 27000000,
        .link_freq = 360000000,
        .reg_list = g_sc450ai_1344x760_4lane_120fps_reg_table,
        .reg_list_size = ARRAY_SIZE(g_sc450ai_1344x760_4lane_120fps_reg_table),
        .hdr_mode = NO_HDR,
    },
    {/* */},
}
```

4.3.1.1.3 struct xxx_dev

【说明】

Camera设备的主要数据结构，包含设备的配置信息、状态信息和操作函数指针。

在rk_xxx_init函数中配置。

【定义】

```
struct xxx_dev
{
    struct rk_camera_device parent;
    char name[RK_CAMERA_DEVICE_NAME_SIZE];
    uint8_t time_valid_delay;
    uint8_t gain_valid_delay;
    char i2c_name[RK_CAMERA_I2C_NAME_SIZE];
    struct rt_i2c_bus_device *i2c_bus;
    uint8_t i2c_addr;
    uint8_t flip;
    struct rt_mutex mutex_lock;
    struct rk_camera_exp_val init_exp;
    uint8_t modes_num;
    const struct sc450ai_mode *supported_modes;
    const struct sc450ai_mode *cur_mode;
    const struct sc450ai_mode *pdst_mode;
    struct sc450ai_mode dst_mode;
    bool has_init_exp;
    bool streaming;
};
```

【成员】

成员名称	描述
parent	Camera设备的父设备
name[RK_CAMERA_DEVICE_NAME_SIZE]	Camera名称
time_valid_delay	曝光时间在N + time_valid_delay帧生效
gain_valid_delay	曝光增益在N + gain_valid_delay帧生效
i2c_name[RK_CAMERA_I2C_NAME_SIZE]	i2c总线名称
*i2c_bus	i2c总线设备
i2c_addr	i2c地址
flip	翻转/镜像设置
mutex_lock	互斥锁
init_exp	初始曝光值
modes_num	初始化配置数量
*supported_modes	根据lane数记录对应模式列表//todo
*cur_mode	当前模式参数
*dst_mode	目标模式参数
has_init_exp	记录是否已经设置初始曝光
streaming	记录设备是否处在出流状态

4.3.1.2 API描述

4.3.1.2.1 rk_xxx_control

【说明】

该函数是一个通用的控制接口，用于应用/ISP下发各种设备控制命令。

【语法】

```
ret_err_t rk_sc450ai_control(struct rk_camera_device *dev,
                             dt_cmd_t cmd,
                             void *args);
```

【参数】

参数名称	描述	输入输出
`*dev`	Camera设备描述符	输入
`cmd`	控制Camera设备的命令	输入
`*args`	命令的参数	输入

【返回值】

返回值	描述
0	成功
非0	失败

【示例】

```
ret_err_t rk_sc450ai_control(struct rk_camera_device *dev,
                             dt_cmd_t cmd,
                             void *args)
{
    ...
    switch (cmd)
    {
    case RK_DEVICE_CTRL_DEVICE_INIT:
    {
        ret = rk_sc450ai_init(dev);
    }
    break;
    ...
    default:
        sc450ai_dbg("exit CMD %d\n", cmd);
        break;
    }

    return ret;
}
```

4.3.1.2.2 rk_xxx_init

【说明】

控制器通过这个接口获取当前 Camera使用的总线参数 ， 来确认控制器的采集方式。

【语法】

```
ret_err_t rk_xxx_init(struct rk_camera_device *dev);
```

【参数】

参数名称	描述	输入输出
*dev	Camera设备描述符	输入

【返回值】

返回值	描述
0	成功
非0	失败

4.3.1.2.3 xxx_stream_on

【说明】

配置Camera初始化寄存器，并根据RT_USING_CAM_STREAM_ON_LATE宏判断是否要提前开启数据流。

【语法】

```
static void xxx_stream_on(struct xxx_dev *dev);
```

【参数】

参数名称	描述	输入输出
*dev	Camera设备描述符	输入

【返回值】

返回值	描述
0	成功
非0	失败

4.3.1.2.4 xxx_stream_off

【说明】

控制Camera关闭数据流。

【语法】

```
static void xxx_stream_off(struct xxx_dev *dev);
```

【参数】

参数名称	描述	输入输出
*dev	Camera设备描述符	输入

【返回值】

返回值	描述
0	成功
非0	失败

4.3.1.2.5 rk_xxx_get_expinf

【说明】

获取Camera曝光信息。

【语法】

```
static ret_err_t rk_xxx_get_expinf(struct xxx_dev *dev, struct rk_camera_exp_info *exp);
```

【参数】

参数名称	描述	输入输出
*dev	Camera设备描述符	输入
*exp	Camera曝光信息相关参数	输出

【返回值】

返回值	描述
0	成功
非0	失败

4.3.1.2.6 rk_xxx_set_expval

【说明】

设置Camera的曝光时间以及曝光增益。

【语法】

```
static ret_err_t rk_xxx_set_expval(struct xxx_dev *dev, struct rk_camera_exp_val *exp);
```

【参数】

参数名称	描述	输入输出
*dev	Camera设备描述符	输入
*exp	Camera曝光信息相关参数	输入

【返回值】

返回值	描述
0	成功
非0	失败

4.3.1.2.7 rk_xxx_set_vts

【说明】

设置Camera VTS。

【语法】

```
static ret_err_t rk_xxx_set_vts(struct xxx_dev *dev, uint32_t dst_vts);
```

【参数】

参数名称	描述	输入输出
*dev	Camera设备描述符	输入
dst_vts	目标VTS值	输入

【返回值】

返回值	描述
0	成功
非0	失败

4.3.1.2.8 rk_xxx_get_intput_fmt

【说明】

获取Camera输出格式，获取到的是当前使用的配置，如前面描述的 struct xxx_mode 里面可能配置多个不同分辨率的配置，驱动初始化的时候会选中其中一个，或者 set_intput_fmt 切换后，可通过 get_intput_fmt 来确认当前的 format。

【语法】

```
static ret_err_t rk_xxx_get_intput_fmt(struct xxx_dev *dev, struct rk_camera_mbus_framefmt
*mbus_fmt);
```

【参数】

参数名称	描述	输入输出
*dev	Camera设备描述符	输入
*mbus_fmt	Camera的输出格式	输出

【返回值】

返回值	描述
0	成功
非0	失败

4.3.1.2.9 rk_xxx_set_intput_fmt

【说明】

设置Camera输出格式，并配置控制器采集参数。

【语法】

```
static ret_err_t rk_xxx_set_intput_fmt(struct xxx_dev *dev, struct rk_camera_mbus_framefmt
*mbus_fmt);
```

【参数】

参数名称	描述	输入输出
*dev	Camera设备描述符	输入
*mbus_fmt	Camera的输出格式	输入

【返回值】

返回值	描述
0	成功
非0	失败

4.3.1.2.10 rk_xxx_match_dst_config

【说明】

根据目标的配置，遍历所有supported_modes，寻找对应目标分辨率图像的输出格式，并判断是否有降帧，如果有降帧，则修改目标VTS。

【语法】

```
static ret_err_t rk_xxx_match_dst_config(struct xxx_dev *dev, struct rk_camera_dst_config
*dst_config);
```

【参数】

参数名称	描述	输入输出
*dev	Camera设备描述符	输入
*dst_config	目标配置的输出格式	输入

【返回值】

返回值	描述
0	成功
非0	失败

4.3.1.2.11 rk_xxx_set_flip_mirror

【说明】

设置Camera翻转或者镜像。

【语法】

```
static ret_err_t rk_xxx_set_flip_mirror(struct xxx_dev *dev, uint32_t flip)
```

【参数】

参数名称	描述	输入输出
*dev	Camera设备描述符	输入
flip	0: 无翻转无镜像 1: 仅翻转 2: 仅镜像 3: 翻转和镜像	输入

【返回值】

返回值	描述
0	成功
非0	失败

4.3.1.2.12 xxx_stream_on_late

【说明】

设置Camera初始曝光、镜像、翻转、开启数据流。

【语法】

```
static void xxx_stream_on_late(struct xxx_dev *dev);
```

【参数】

参数名称	描述	输入输出
*dev	Camera设备描述符	输入

【返回值】

返回值	描述
0	成功
非0	失败

4.3.1.2.13 xxx_quick_stream

【说明】

该函数用于开启、关闭Camera数据流。

【语法】

```
static void xxx_quick_stream(struct xxx_dev *dev, int on);
```

【参数】

参数名称	描述	输入输出
*dev	Camera设备描述符	输入
on	1: 启动stream输出; 0: 停止stream输出	输入

【返回值】

返回值	描述
0	成功
非0	失败

4.3.1.2.14 rk_camera_xxx_init

【说明】

该函数用于Sensor I2C设备注册，需要参考Sensor Datasheet中的上电时序进行配置。

【语法】

```
int rk_xxx_check_sensor_id(struct xxx_dev *dev)
```

【参数】

参数名称	描述	输入输出
*dev	Camera设备描述符	输入

【返回值】

返回值	描述
0	成功
非0	失败

4.3.2 摄像头驱动移植

编写新的 Sensor 驱动时，可基于一款规格接近的 Sensor （`rt-thread/bsp/rockchip/camera/`) 驱动或相同厂家型号相近的驱动修改 。

4.3.2.1 添加编译配置选项

在rt-thread/bsp/rockchip/common/drivers/Kconfig添加如下修改。

```
@@ -506,6 +506,8 @@ menu "supported camera"
        bool "Enable GC2053"
    config RT_USING_F37
        bool "Enable F37"
    config RT_USING_SC230AI
        bool "Enable SC230AI"
+   config RT_USING_SC450AI
+       bool "Enable SC450AI"
    config RT_USING_SC850SL
        bool "Enable SC850SL"
```

4.3.2.2 寄存器序列转换

快启模式下，MCU Sensor驱动均采用I2C连续写的方式写入初始化配置，原厂提供的配置一般为非连续的格式，需要使用脚本转换成驱动对应的连续写格式。

4.3.2.2.1 序列转换步骤

1. 需要参考`<sdk>/sysdrv/source/mcu/project/scripts/camera_reg_convert/reg.txt`中的格式修改初始化序列。
2. 将初始化序列替换至`<sdk>/sysdrv/source/mcu/project/scripts/camera_reg_convert/reg.txt`中。
3. 使用如下命令进行转换：

```
cd <sdk>/sysdrv/source/mcu/project/scripts/camera_reg_convert/
python2 reg_conver.py
```

4. 将转换的结果复制替换到Sensor驱动中的对应寄存器序列中。

【示例】

```
/*
 * Xclk 27Mhz
 * max_framerate 30fps
 * mipi_datarate per lane 720Mbps, 4lane
 * full resolution
 */
static const uint8_t g_sc450ai_2688x1520_4lane_30fps_reg_table[] =
{
    0x3, 0x01, 0x00, 0x00,
    0x3, 0x36, 0xe9, 0x80,
    0x3, 0x36, 0xf9, 0x80,
    0x3, 0x30, 0x1c, 0x78,
    0x3, 0x30, 0x1f, 0x01,
    0x3, 0x30, 0x2e, 0x00,
    0xa, 0x32, 0x08, 0x0a, 0x80, 0x05, 0xf0, 0x02, 0xee, 0x06, 0x18,
    0x4, 0x32, 0x14, 0x11, 0x11,
    0x8,
};
```

【说明】

连续写格式说明

第1列为单次写入的I2C DATA字节数。

第2-3列为I2C寄存器地址。

第4-n列为连续写入的I2C寄存器值。

最后一行为发起I2C写的总次数。

【注意事项】

寄存器序列中不要开启出流寄存器，在xxx_stream_on_late函数中单独开启。

4.3.2.3 配置基础函数

实现rk_camera_ops中所有函数，包含rk_xxx_control中的所有control。

实现rk_camera_xxx_init函数。

4.3.3 MCU板级配置说明

4.3.3.1 CIS(cmos image sensor)设备注册

4.3.3.1.1 struct gpio_desc

【定义】

```
struct gpio_desc
{
    uint32_t gpio_pin;
    struct GPIO_REG *gpio_group;
};
```

4.3.3.1.2 camera_board_desc 结构体说明

【描述】

Sensor与板级硬件相关的配置信息

【定义】

```
struct camera_board_desc
{
    struct gpio_desc rst_gpio;
    struct gpio_desc pwdn_gpio;
    struct gpio_desc pwren_gpio;
    eCLOCK_Name mclk_id;
    uint32_t mclk_out_gate_id;
    char isp_subdev_name[RK_CAMERA_DEVICE_NAME_SIZE];
    char i2c_bus[RK_CAMERA_I2C_NAME_SIZE];
    uint8_t i2c_addr;
    uint8_t time_valid_delay;
    uint8_t gain_valid_delay;
    uint8_t mode_id;
    uint32_t width;
    uint32_t height;
    uint32_t pixelcode;
    uint32_t maxfps;
    uint32_t lanes;
    uint32_t hdr_mode;
    uint32_t dst_width;
    uint32_t dst_height;
    uint32_t dst_pixelcode;
    uint32_t dst_maxfps;
    uint32_t dst_hdr_mode;
    bool rst_active;
    bool pwdn_active;
    bool pwren_active;
};
```

【成员】

成员名称	描述
rst_gpio	硬件复位引脚配置, 可以根据gpio_desc的定义实现。
pwdn_gpio	低功耗模式引脚配置,可以根据gpio_desc的定义实现。
pwren_gpio	上电引脚配置,可以根据gpio_desc的定义实现。
mclk_id	通过配置0或者1选择使用mclk0 或者mclk1。
isp_subdev_name	Sensor是否作为主摄连接到ISP。主摄：RK_ISP_MAIN_SUBDEV_NAME 辅摄：RK_ISP_ASSIST_SUBDEV_NAME
i2c_bus	选择Sensor I2C总线
i2c_addr	配置Sensor I2C地址（七位地址）
time_valid_delay	曝光时间生效所需要的帧数
gain_valid_delay	曝光增益生效所需要的帧数
mode_id	选择Sensor分辨率，可以根据camera.h里面的分辨率配置
width	MCU小图的宽度
height	MCU小图的高度
pixelcode	MCU小图图像格式
maxfps	MCU小图所有初始化配置中的最大帧率
lanes	Sensor lane数
hdr_mode	MCU小图HDR模式 (NO_HDR: 0 HDR_X2: 5 HDR_X3: 6)
dst_width	MCU大图的宽度
dst_height	MCU大图的高度
dst_pixelcode	MCU大图图像格式
dst_maxfps	MCU大图所有初始化配置中的最大帧率
dst_hdr_mode	MCU大图HDR模式 (NO_HDR: 0 HDR_X2: 5 HDR_X3: 6)
rst_active	硬件复位引脚的有效电平
pwdn_active	低功耗模式引脚的有效电平
pwren_active	上电引脚的有效电平

4.3.3.1.3 cif_hw_config 函数说明

data_lanes：sensor lane数

Sensor、csi_dphy、csi_host连接关系如下表

主摄与dphy硬件连接关系	dphy1.enable	dphy2.enable	csi_host_0.enable	csi_host_1.enable
lane0 < - > MIPI_CSI_RX_D0N/P	1	0	1	0
lane1 < - > MIPI_CSI_RX_D1N/P	1	0	1	0
lane2 < - > MIPI_CSI_RX_D2N/P	0	1	0	1
lane3 < - > MIPI_CSI_RX_D3N/P	0	1	0	1

4.3.3.1.4 单摄注册

【示例】

```
const struct camera_board_desc camera_sc450ai =
{
    .i2c_addr = 0x30,
    .rst_gpio =
    {
        .gpio_pin = GPIO_PIN_A7,
        .gpio_group = GPIO4,
    },
    .pwdn_gpio =
    {
        .gpio_pin = GPIO_PIN_A2,
        .gpio_group = GPIO4,
    },
    .time_valid_delay = 2,
    .gain_valid_delay = 2,
    .width = 1344,
    .height = 760,
    .maxfps = 120,
    .lanes = 4,
    .pixelcode = MEDIA_BUS_FMT_SBGGR10_1X10,
    .hdr_mode = NO_HDR,
    .dst_width = 2688,
    .dst_height = 1520,
    .dst_maxfps = 60,
    .dst_pixelcode = MEDIA_BUS_FMT_SBGGR10_1X10,
    .dst_hdr_mode = NO_HDR,
    .isp_subdev_name = RK_ISP_MAIN_SUBDEV_NAME,
    .i2c_bus = "i2c3",
    .rst_active = 0,
    .pwdn_active = 1,
    .mclk_id = CLK_MIPI0_OUT2IO,
    .mclk_out_gate_id = CLK_MIPI0_OUT2IO_GATE,
};
```

```
void cif_hw_config(void)
{
    struct dphy_board_desc *csi2_dphy = (struct dphy_board_desc *)&csi2_dphy_board;
    struct csi2_board_desc *csi_host_0 = (struct csi2_board_desc *)&csi2_host0_board;
    struct vicap_board_desc *vicap = (struct vicap_board_desc *)&vicap_board;
```

```
    csi2_dphy->enable = true;
    csi2_dphy->csi2_dphy1.enable = true;
    csi2_dphy->csi2_dphy1.csi_host_idx = 0;
    csi2_dphy->csi2_dphy1.isp_subdev_name = RK_ISP_MAIN_SUBDEV_NAME;

    csi_host_0->enable = true;

    vicap->enable = true;
}
```

4.3.4 如何新增板级配置

1. 在`bsp/rockchip/xxx_mcu/board/xxx/board.c`中实现camera_board_desc结构体和cif_hw_config函数。参照CIS设备注册
2. 在`bsp/rockchip/xxx_mcu/board/xxx/iomux.c`中，根据硬件原理图添加外设iomux配置。

5. 新的功能支持

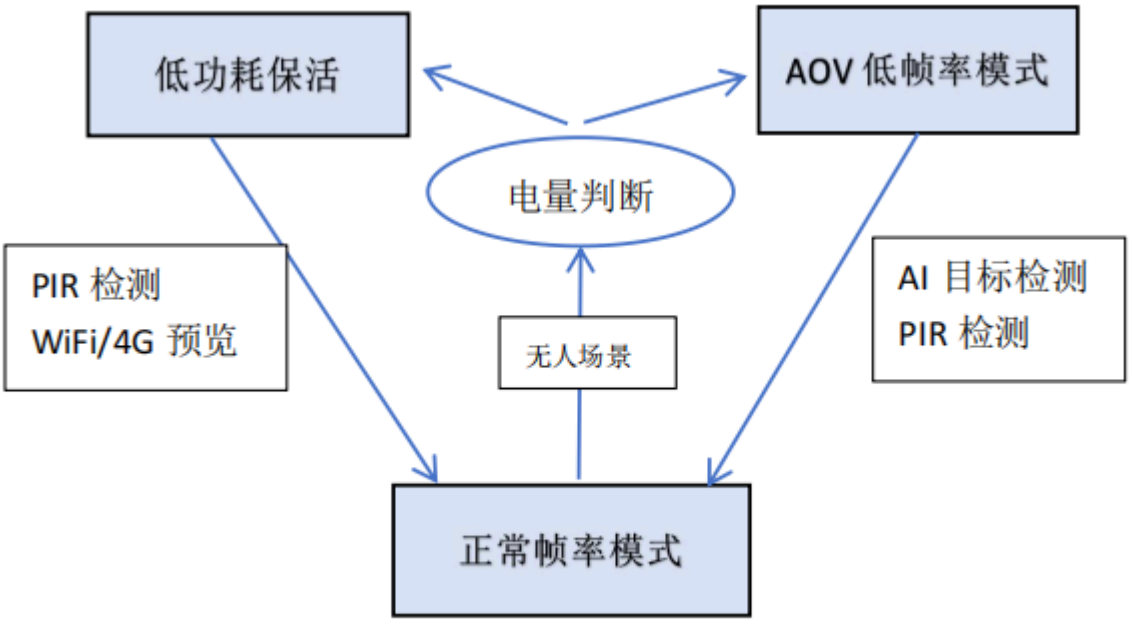
5.1 AOV

AOV (Always On Video)基于休眠唤醒实现的24小时低帧率持续录像，有事件时也可以切回正常帧率模式，如AI检测、PIR触发、WIFI唤醒等。

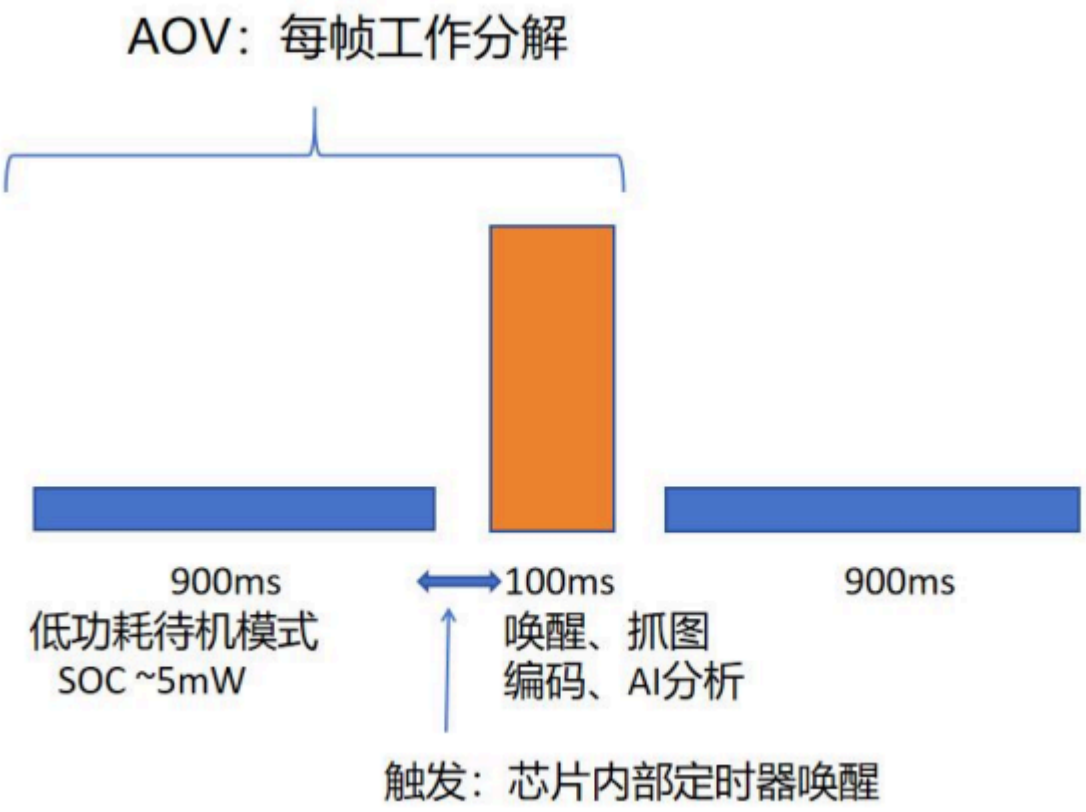
5.1.1 场景说明

三个场景:

- 正常帧率模式：远程预览、PIR 事件、人形检测事件时进入。需要录像、联网等功能；
- AOV 低帧率模式：空闲时，电量足，SoC 低帧率主码流录像、WiFi/4G 保活；
- 低功耗保活模式：空闲时，电量较低时，SoC 下电、WiFi/4G 保活。


低功耗保活
AOV 低帧率模式
电量判断
PIR 检测
WiFi/4G 预览
无人场景
AI 目标检测
PIR 检测
正常帧率模式

5.1.2 工作流程


AOV：每帧工作分解
900ms
低功耗待机模式
SOC ~5mW
100ms
唤醒、抓图
编码、AI分析
900ms
触发：芯片内部定时器唤醒

5.1.3 概念说明

AOV 模式涉及Sensor的 standby 模式。

standby 模式是指 Sensor 为了节省功耗处在睡眠模式或者复位模式。

- 睡眠模式

  Sensor 停止图像数据流，工作在低功耗状态，保持当前寄存器值。
  以 SC200AI 为例，有两种方式进入睡眠模式：

  1. 将 PWDN 拉低，此时不支持 I2C 的读写。
  2. 将出流控制寄存器写入0，此时支持 I2C 读写。

- 复位模式

  Sensor 停止图像数据流，工作在低功耗状态，重置所有寄存器。

  以 SC200AI 为例，有两种方式进入复位模式：

  1. 将 XSHUTDN 拉低，此时不支持 I2C 读写。
  2. 将软使能复位寄存器写入1，此复位模式持续 150ns。

5.1.3.1 硬件 standby 模式

`硬件 standby 模式`指通过拉低 PWDN 管脚，使 Sensor 进入睡眠模式，从而使其工作在低功耗状态，此时 I2C 不可读写。

以 SC200AI 为例，DTS 中 , Sensor 节点添加支持硬件 standby 模式属性 rockchip,camera-module-stb = <1> ，其属性值用来表示是否支持硬件 standby 方式，1 表示指支持，0 表示不支持。

```
sc200ai: sc200ai@30 {
    compatible = "smartsens,sc200ai";
    status = "okay";
    reg = <0x30>;
    ...
    rockchip,camera-module-stb = <1>;
    port {
        sc200ai_out: endpoint {
            remote-endpoint = <&csi_dphy_input1>;
            data-lanes = <1 2>;
        };
    };
};
```

Sensor 驱动 probe 接口中需添加解析该属性值，获取到硬件 standby 模式是否支持，并保存在 sc200ai->standby_hw 成员变量中。

若支持硬件standby 模式，ioctl 处理接口中， RKMODULE_SET_QUICK_STREAM 命令会根据起流状态，进行上、下拉相关 PWDN GPIO。休眠唤醒中会下发 v4l2 ioctl 命令来重新设置曝光、增益寄存器等。相关驱动参考 4.2.4 驱动实现 硬件 standby 驱动参考代码。

5.1.3.2 软件 standby 模式

`软件 standby 模式`是指在休眠唤醒后，通过写入完整 Sensor 寄存器配置的方式，然后在 Sensor ioctl 接口中，RKMODULE_SET_QUICK_STREAM 命令处理时，直接配置起流寄存器的休眠唤醒方式。相关驱动参考 4.2.4 驱动实现 软件 standby 驱动参考代码。

5.1.4 驱动实现

硬件 standby 模式驱动参考代码：

```
static long sc200ai_ioctl(struct v4l2_subdev *sd, unsigned int cmd, void *arg)
{
    ......
    switch (cmd) {
    case RKMODULE_SET_QUICK_STREAM:
            stream = *((u32 *)arg);

            if (sc200ai->standby_hw) { // hardware standby
                if (stream) {
                    if (!IS_ERR(sc200ai->pwdn_gpio))
                        gpiod_set_value_cansleep(sc200ai->pwdn_gpio, 1);
                    // Make sure __v4l2_ctrl_handler_setup can be called correctly
                    sc200ai->is_standby = false;
                    ret = sc200ai_write_reg(sc200ai->client, SC200AI_REG_MIPI_CTRL,
                        SC200AI_REG_VALUE_08BIT, SC200AI_MIPI_CTRL_ON);

                    #if IS_REACHABLE(CONFIG_VIDEO_CAM_SLEEP_WAKEUP)
                    if (__v4l2_ctrl_handler_setup(&sc200ai->ctrl_handler))
                        dev_err(&sc200ai->client->dev, "__v4l2_ctrl_handler_setup fail!");
                    if (sc200ai->cur_mode->hdr_mode != NO_HDR) {
                        if (sc200ai->cam_sw_inf) {
                            ret = sc200ai_ioctl(&sc200ai->subdev,
                                        PREISP_CMD_SET_HDRAE_EXP,
                                        &sc200ai->cam_sw_inf->hdr_ae);
                            if (ret) {
                                dev_err(&sc200ai->client->dev,
                                    "init exp fail in hdr mode\n");
                                return ret;
                            }
                        }
                    }
                    #endif

                    ret |= sc200ai_write_reg(sc200ai->client, SC200AI_REG_CTRL_MODE,
                        SC200AI_REG_VALUE_08BIT, SC200AI_MODE_STREAMING);

                    dev_info(&sc200ai->client->dev, "quickstream, streaming on: exit hw
standby mode\n");
                } else {
                    ret = sc200ai_write_reg(sc200ai->client, SC200AI_REG_CTRL_MODE,
```

```
                        SC200AI_REG_VALUE_08BIT, SC200AI_MODE_SW_STANDBY);

                    ret |= sc200ai_write_reg(sc200ai->client, SC200AI_REG_MIPI_CTRL,
                        SC200AI_REG_VALUE_08BIT, SC200AI_MIPI_CTRL_OFF);

                    if (!IS_ERR(sc200ai->pwdn_gpio))
                        gpiod_set_value_cansleep(sc200ai->pwdn_gpio, 0);

                    dev_info(&sc200ai->client->dev, "quickstream, streaming off: enter hw
standby mode\n");
                    sc200ai->is_standby = true;
                }
            }
        ......
}
```

软件 standby 驱动参考实现:

```
static long sc200ai_ioctl(struct v4l2_subdev *sd, unsigned int cmd, void *arg)
{
    ......
    switch (cmd) {
    case RKMODULE_SET_QUICK_STREAM:
            stream = *((u32 *)arg);
            ......
            else {  // software standby
            if (stream) {
                ret = sc200ai_write_reg(sc200ai->client, SC200AI_REG_MIPI_CTRL,
                    SC200AI_REG_VALUE_08BIT, SC200AI_MIPI_CTRL_ON);

                ret |= sc200ai_write_reg(sc200ai->client, SC200AI_REG_CTRL_MODE,
                    SC200AI_REG_VALUE_08BIT, SC200AI_MODE_STREAMING);
                dev_info(&sc200ai->client->dev, "quickstream, streaming on: exit soft
standby mode\n");
            } else {
                ret = sc200ai_write_reg(sc200ai->client, SC200AI_REG_CTRL_MODE,
                    SC200AI_REG_VALUE_08BIT, SC200AI_MODE_SW_STANDBY);

                ret |= sc200ai_write_reg(sc200ai->client, SC200AI_REG_MIPI_CTRL,
                    SC200AI_REG_VALUE_08BIT, SC200AI_MIPI_CTRL_OFF);
                dev_info(&sc200ai->client->dev, "quickstream, streaming off: enter soft
standby mode\n");
            }
        }

        break;
        ......
    }
```

5.1.5 参考文档

AOV功能开发过程中，可以参考以下相关文档：

- Rockchip_Trouble_Shooting_Linux5.10_Camera_CN.pdf
- Rockchip_RV1106_Developer_Guide_Linux_AOV_CN.pdf

5.2 同步模式 (主从模式)

单个Sensor 控制模式比较简单，一般都是 Master 模式。但对两个或两个以上Sensor来说，如果需要做同步出流、同步曝光等操作时，就需要设计到控制模式的配置。一般根据情况配置为主从模式。

在以下应用中，我们常常需要 **多个摄像头 Sensor 同时曝光、同时输出一帧图像，保证多路图像在时间上严格对齐**：

应用场景	**是否需要多 Sensor 同步?**
3D 感知 / 双目视觉 / 立体视觉	必须同步，否则深度计算错误
多摄像头拼接 / 全景成像	需要帧同步，图像才能精准对齐
机器视觉 / 工业检测	多视角同步采样
自动驾驶 / ADAS（多摄像头输入）	多路摄像头时间同步非常关键
视频会议 / 多目摄像头	⚠通常不需要严格同步，但同步可提升体验

实现多 Sensor 同步的常见方法就是：

使用一个 **全局同步信号（如 FSYNC / EFSYNC / TRIGGER / SYNC_IN）**，由主控芯片（如 SoC）统一发送触发信号，在同一时刻让多个 Sensor **同时开始曝光、读出帧数据**，从而保证多路图像在时间维度上严格同步。

5.2.1 Sensor 的 Slave模式

在 **图像传感器（如 CMOS Sensor）** 的通信与控制模式中，**Slave 模式（从模式）** 是指：该 Sensor 的通信、时序控制、数据传输等工作方式是由外部主机（如 MCU、SoC、DSP 等）通过外部信号（如时钟、同步信号、I2C / MIPI 控制）来驱动和控制的，Sensor 自身不主动发起通信或控制时序，而是被动响应主机的指令。

什么是 FSYNC / EFSYNC?

在多 Sensor 系统中，主控芯片（如 SoC / FPGA）会在同一时刻向所有 Sensor 发送 **一个 FSYNC 脉冲**，所有 Sensor 收到该信号后：

- **同时开始曝光**
- 或 **同时开始读出一帧数据**
- 从而保证多路图像的时间戳一致、内容同步

同步信号名称	含义	说明
FSYNC (Frame SYNC)	帧同步信号	一个 **外部输入的数字信号（通常为 GPIO / CMOS 电平）**，用于通知多个 Sensor **在同一时刻开始新的一帧曝光或读出**
EFSYNC (External Frame SYNC)	外部帧同步信号（部分厂商命名不同）	和 FSYNC 类似，是 **由主控统一发出，所有 Slave Sensor 接收并同步动作** 的信号
TRIGGER / SYNC_IN / STROBE	某些 Sensor 的叫法	功能类似，都是用于外部触发的同步输入信号

RK 多摄的同步模式：

目前配置一个Sensor工作在Master模式，其他Sensor工作在Slave模式。等作为Master的Sensor出帧完成后，通过外部同步信号（**FSYNC / EFSYNC**）的到来，然后开始作为Slave Sensor同时开始一帧的曝光和读出过程，从而保证多路图像的时间对齐。

5.2.2 驱动实现

1. 根据原厂 Datasheet 或找原厂 FAE 索要主从模式寄存器配置。

```
/* sync mode regs */
static __maybe_unused const struct regval sc200ai_interal_sync_master_start_regs[] = {
    {0x300a, 0x24}, //sync as output PAD
    {0x3032, 0xa0},
    {0x3222, 0x00}, //master mode
    {0x3216, 0x00},
    {0x3217, 0x04},
    {REG_NULL, 0x00},
};

static __maybe_unused const struct regval sc200ai_interal_sync_master_stop_regs[] = {
    {REG_NULL, 0x00},
};

static __maybe_unused const struct regval sc200ai_interal_sync_slaver_start_regs[] = {
    {0x3222, 0x02}, //slave mode
    {0x3230, 0x00},
    {0x3231, 0x04}, //Rows Before Read
    {0x3224, 0x82}, //fsync trigger
    {REG_NULL, 0x00},
};

static __maybe_unused const struct regval sc200ai_interal_sync_slaver_stop_regs[] = {
    {REG_NULL, 0x00},
};
```

2. 添加同步模式变量

```
struct sc200ai {
    ......
    enum rkmodule_sync_mode sync_mode;
    ......
};
```

常用同步模式定义：

```
/*
 * sensor exposure sync mode
 */
enum rkmodule_sync_mode {
    NO_SYNC_MODE = 0,
    EXTERNAL_MASTER_MODE,
    INTERNAL_MASTER_MODE,
    SLAVE_MODE,
    SOFT_SYNC_MODE,
};
```

同步模式的解析：

解析DTS设备树中相关 Sensor 节点的同步模式属性值，来确定当前Sensor工作在哪个同步模式。

```
static int sc200ai_probe(struct i2c_client *client,
             const struct i2c_device_id *id)
    ......
    ret = of_property_read_string(node, RKMODULE_CAMERA_SYNC_MODE,
                      &sync_mode_name);
    if (ret) {
        sc200ai->sync_mode = NO_SYNC_MODE;
        dev_err(dev, "could not get sync mode!\n");
    } else {
        if (strcmp(sync_mode_name, RKMODULE_EXTERNAL_MASTER_MODE) == 0) {
            sc200ai->sync_mode = EXTERNAL_MASTER_MODE;
            dev_info(dev, "sync_mode = [EXTERNAL_MASTER_MODE]\n");
        } else if (strcmp(sync_mode_name, RKMODULE_INTERNAL_MASTER_MODE) == 0) {
            sc200ai->sync_mode = INTERNAL_MASTER_MODE;
            dev_info(dev, "sync_mode = [INTERNAL_MASTER_MODE]\n");
        } else if (strcmp(sync_mode_name, RKMODULE_SLAVE_MODE) == 0) {
            sc200ai->sync_mode = SLAVE_MODE;
            dev_info(dev, "sync_mode = [SLAVE_MODE]\n");
        } else if (strcmp(sync_mode_name, RKMODULE_SOFT_SYNC_MODE) == 0) {
            sc200ai->sync_mode = SOFT_SYNC_MODE;
            dev_info(dev, "sync_mode = [SOFT_SYNC_MODE]\n");
        } else {
            dev_info(dev, "sync_mode = [NO_SYNC_MODE]\n");
        }
    }
    ......
```

相关宏定义：

```
#define RKMODULE_CAMERA_SYNC_MODE       "rockchip,camera-module-sync-mode"
#define RKMODULE_INTERNAL_MASTER_MODE   "internal_master"
#define RKMODULE_EXTERNAL_MASTER_MODE   "external_master"
#define RKMODULE_SLAVE_MODE             "slave"
#define RKMODULE_SOFT_SYNC_MODE         "soft_sync"
```

3. 根据当前Sensor工作的同步模式起流时配置不同的寄存器配置

```
static int __sc200ai_start_stream(struct sc200ai *sc200ai)
{
    int ret = 0;
    ......
    if (!sc200ai->is_thunderboot) {
        ret = sc200ai_write_array(sc200ai->client, sc200ai->cur_mode->reg_list);
        if (ret)
            return ret;

        ......

        if (sc200ai->sync_mode == INTERNAL_MASTER_MODE ||
            sc200ai->sync_mode == SOFT_SYNC_MODE) {
            ret |= sc200ai_write_array(sc200ai->client,
                           sc200ai_interal_sync_master_start_regs);
        } else if (sc200ai->sync_mode == EXTERNAL_MASTER_MODE) {
            ret |= sc200ai_write_array(sc200ai->client,
                           sc200ai_interal_sync_slaver_start_regs);
        } else if (sc200ai->sync_mode == SLAVE_MODE) {
            ret |= sc200ai_write_array(sc200ai->client,
                           sc200ai_interal_sync_slaver_start_regs);
        }
        if (ret) {
            dev_err(&sc200ai->client->dev,
                "write sync regs failed\n");
            return ret;
        }
    }

    if (sc200ai->sync_mode == NO_SYNC_MODE)
        ret |= sc200ai_write_reg(sc200ai->client, SC200AI_REG_CTRL_MODE,
                     SC200AI_REG_VALUE_08BIT, SC200AI_MODE_STREAMING);
    return ret;
}
```

4. RKAIQ 中设置/获取同步模式

若设备树中，没有配置相关Sensor的同步工作模式，调用rkaiq 接口也可以通过 ioctl 命令设置多摄的同步工作模式。

```
static long sc200ai_ioctl(struct v4l2_subdev *sd, unsigned int cmd, void *arg)
{
    ......
    case RKMODULE_GET_SYNC_MODE:
        sync_mode = (u32 *)arg;
```

```
        *sync_mode = sc200ai->sync_mode;
        break;
    case RKMODULE_SET_SYNC_MODE:
        sync_mode = (u32 *)arg;
        if (sync_mode) {
            sc200ai->sync_mode = *sync_mode;
            dev_info(&sc200ai->client->dev, "set sync mode is: %s\n",
                 ((*sync_mode == EXTERNAL_MASTER_MODE) ||
                  (*sync_mode == SLAVE_MODE)) ? "secondary" : "primary");
        } else {
            dev_info(&sc200ai->client->dev, "set sync mode is: NO_SYNC_MODE\n");
        }
        break;
    ......
}
```

5.3 闪光灯控制

Sensor 驱动中可以实现对闪光灯的控制，支持的闪光灯控制类型有 PWM, GPIO。应用层控制闪光灯开启、关闭流程。需要正确配置相关参数。

5.3.1 驱动实现

1. 头文件

```
#include "light_ctl.h"
```

2. 添加相关控制变量

```
struct xxxx {
    ......
    bool                    enable_light_ctl;
    struct rk_light_param   light_param;
    ......
};
```

3. 添加ioctl RKCIS_CMD_FLASH_LIGHT_CTRL命令处理

```
static long xxxx_ioctl(struct v4l2_subdev *sd, unsigned int cmd, void *arg)
    ......
    struct rk_light_param *light_param;
    ......
    case RKCIS_CMD_FLASH_LIGHT_CTRL:
        dev_info(&mis5001->client->dev, "set flash light param\n");
        light_param = (struct rk_light_param *)arg;
        if (light_param->light_enable) {
            memcpy(&mis5001->light_param, light_param, sizeof(struct rk_light_param));
            mis5001->enable_light_ctl = true;
```

```
        } else {
            mis5001->enable_light_ctl = false;
        }
        ret = light_ctl_write(mis5001->module_index, light_param);
        break;

    ......
};

static long xxxx_compat_ioctl32(struct v4l2_subdev *sd,
                  unsigned int cmd, unsigned long arg)
{
        ......
        struct rk_light_param *light_param;
        ......
        switch (cmd) {
        ......
        case RKCIS_CMD_FLASH_LIGHT_CTRL:
            light_param = kzalloc(sizeof(*light_param), GFP_KERNEL);
            if (!light_param) {
                ret = -ENOMEM;
                return ret;
            }

            ret = copy_from_user(light_param, up, sizeof(*light_param));
            if (!ret)
                ret = mis5001_ioctl(sd, cmd, light_param);
            else
                ret = -EFAULT;
            kfree(light_param);
            break;
        }
        .....
}
```

5.3.2 参考文档

具体添加与实现，可参考如下文档：

`Rockchip_Trouble_Shooting_Linux5.10_Camera_CN.pdf` 章节**7.2 如何对补光灯进行控制**。

5.4 分辨率切换

客户应用中可能在不同场景下使用不同Sensor的分辨率，为了兼容这种情况，Sensor驱动支持上层应用的命令响应，切换Sensor的分辨率。

5.4.1 驱动实现

驱动代码参考：

```
static int sc200ai_set_setting(struct sc200ai *sc200ai, struct rk_sensor_setting *setting)
{
    int i = 0;
    int cur_fps = 0;
    s64 h_blank, vblank_def;
    u64 pixel_rate = 0;
    const struct sc200ai_mode *mode = NULL;
    const struct sc200ai_mode *match = NULL;

    dev_info(&sc200ai->client->dev,
        "sensor setting: %d x %d, fps:%d fmt:%d, mode:%d\n",
        setting->width, setting->height,
        setting->fps, setting->fmt, setting->mode);

    for (i = 0; i < ARRAY_SIZE(supported_modes); i++) {
        mode = &supported_modes[i];
        if (mode->width == setting->width &&
            mode->height == setting->height &&
            mode->hdr_mode == setting->mode &&
            mode->bus_fmt == setting->fmt) {
            cur_fps = DIV_ROUND_CLOSEST(mode->max_fps.denominator, mode->max_fps.numerator);
            if (cur_fps == setting->fps) {
                match = mode;
                break;
            }
        }
    }

    if (match) {
        dev_info(&sc200ai->client->dev, "-----%s: match the support mode, mode idx:%d-----
\n",
            __func__, i);
        sc200ai->cur_mode = mode;

        h_blank = mode->hts_def - mode->width;
        __v4l2_ctrl_modify_range(sc200ai->hblank, h_blank,
                    h_blank, 1, h_blank);
        vblank_def = mode->vts_def - mode->height;
        __v4l2_ctrl_modify_range(sc200ai->vblank, vblank_def,
                    SC200AI_VTS_MAX - mode->height,
                    1, vblank_def);
        __v4l2_ctrl_s_ctrl(sc200ai->link_freq, mode->mipi_freq_idx);
        pixel_rate = (u32)link_freq_menu_items[mode->mipi_freq_idx] /
                 mode->bpp * 2 * SC200AI_LANES;
        __v4l2_ctrl_s_ctrl_int64(sc200ai->pixel_rate, pixel_rate);
        dev_info(&sc200ai->client->dev, "freq_idx:%d pixel_rate:%lld\n",
            mode->mipi_freq_idx, pixel_rate);
```

```
        sc200ai->cur_vts = mode->vts_def;
        sc200ai->cur_fps = mode->max_fps;

        dev_info(&sc200ai->client->dev, "hts_def:%d cur_vts:%d cur_fps:%d\n",
            mode->hts_def, mode->vts_def,
            sc200ai->cur_fps.denominator / sc200ai->cur_fps.numerator);
    } else {
        dev_err(&sc200ai->client->dev, "couldn't match the support modes\n");
        return -EINVAL;
    }

    return 0;
}

static long sc200ai_ioctl(struct v4l2_subdev *sd, unsigned int cmd, void *arg)
    ......
    struct rk_sensor_setting *setting;
    ......

    switch (cmd) {
    ......
    case RKCIS_CMD_SELECT_SETTING:
        setting = (struct rk_sensor_setting *)arg;
        ret = sc200ai_set_setting(sc200ai, setting);
        break;

    ......
    }
}
```

5.5 多 lane 配置的兼容

Sensor 根据主控平台、个数、性能、连接方式等的不同可能需要兼容不同 Lane 数的 寄存器序列配置。所以驱动中针对此情况做了兼容。

5.5.1 驱动实现

具体代码参考如下：

```
// 4 Lane 寄存器配置支持列表
static const struct sc450ai_mode supported_modes_4lane[] = {
    {
        ...
    },
    {
        ...
    },
};
```

```
// 2 Lane 寄存器配置支持列表
static const struct sc450ai_mode supported_modes_2lane[] = {
    {
        ...
    },
    {
        ...
    },
}
static int sc450ai_probe(struct i2c_client *client,
             const struct i2c_device_id *id)
{
    struct device_node *node = dev->of_node;
    struct device_node *endpoint;
    ......
    endpoint = of_graph_get_next_endpoint(dev->of_node, NULL);
    if (!endpoint) {
        dev_err(dev, "Failed to get endpoint\n");
        return -EINVAL;
    }
    ret = v4l2_fwnode_endpoint_parse(of_fwnode_handle(endpoint),
        &sc450ai->bus_cfg);
    of_node_put(endpoint);
    if (ret) {
        dev_err(dev, "Failed to get bus config\n");
        return -EINVAL;
    }

    // 根据dts配置的lane数选择不同的寄存器支持列表数组
    if (sc450ai->bus_cfg.bus.mipi_csi2.num_data_lanes == 4) {
        sc450ai->supported_modes = supported_modes_4lane;
        sc450ai->cfg_num = ARRAY_SIZE(supported_modes_4lane);
        dev_info(dev, "detect sc450ai lane: %d\n",
             sc450ai->bus_cfg.bus.mipi_csi2.num_data_lanes);
    } else {
        sc450ai->supported_modes = supported_modes_2lane;
        sc450ai->cfg_num = ARRAY_SIZE(supported_modes_2lane);
        dev_info(dev, "detect sc450ai lane: %d\n",
             sc450ai->bus_cfg.bus.mipi_csi2.num_data_lanes);
    }
    ......
}
```

支持2Lane 或 4Lane 的supported_modes，在Sensor的 probe 接口调用时，解析 DTS 中 Sensor 的 Lane 个数来进行初始化, 解析结果保存在bus_cfg.bus.mipi_csi2.num_data_lanes中，并配置每种 Lane modes 中 cfg_num 总数。

5.6 其他

6. 常见问题分析与解决方案

6.1 Sensor不识别，probe 失败，I2C通讯失败

Sensor ID 如果未识别到，这与 RKISP 或 RKCIF 没有任何关系，仅仅是 Sensor 上电时序未满足要求。
请按以下顺序排查:

1. Sensor 的 7-bits i2c slave id 是否正确, 是否误写成8-bits。
2. 检查 MCLK 是否有输出, 电压幅度是否正确。MCLK一般是 24Mhz，也有27Mhz。
3. Sensor 的上电时序是否满足要求，主要包括 avdd, dovdd, dvdd 电压是否正常，是否符合Sensor手册要求, PWDN, RESET gpio管脚是否配置正确等。

6.1.1 测量不到 MCLK 和 VDD 电压

在 Sensor 驱动的实现中，一般是只有在需要时才开启 MCLK 及电源，因此开机后 MCLK 及电源默认是关闭的。

调试时，可以将驱动中的 power_off() 函数的实现注释掉，这样不会下电，方便测量。

使用示波器时，检查示波器的带宽是否足够，建议至少 48M 以上的带宽。

1. Sensor 没有正确打开 MCLK，请参考如 drivers/media/i2c/ov5695.c 中对 MCLK 的操作。
2. 该 gpio 被其它模块占用了，这种情况时，一般 kernel log 会有相应的提示。还可以通过 io 命令去查看 pin-ctrl 寄存器设置是否正确。

6.1.2 检查 Sensor 的上电时序是否满足要求

Sensor 的 Datasheet 中一般会详细描述每路电源的上电顺序及间隔要求，请通过示波器检查是否满足。
有一些 Sensor 的电源 vdd 在上电时是没有时间先后要求的，如 ov5695 ，它的驱动中可能是用 regulator_bulk 来管理电源；但有一些是有先后要求的，如 ov2685.c ，它在驱动中是用多个regulator 去分别控制，具体如 avdd_regulator , dovdd_regulator 。请根据实际情况选择。

6.2 驱动中的 `exp_def` 、 `hts_def` 、 `vts_def` 默认值应该设多少

如果有 Sensor 原厂的联系方式，请联系原厂获取。否则，请从 Datasheet 中查找到对应的寄存器，并从寄存器列表中找到初始化时配置的值即可。以 SC200AI 为例：

```
#define SC200AI_REG_VTS_H       0x320e
#define SC200AI_REG_VTS_L       0x320f
...
```

```
static const struct regval sc200ai_linear_10_1920x1080_30fps_regs[] = {
    ...
    //HTS=1100*2=2200
    {0x320c, 0x04},
    {0x320d, 0x4c},
    //VTS=1125
    {0x320e, 0x04},
    {0x320f, 0x65},
    ...
};

static const struct sc200ai_mode supported_modes[] = {
    {
        .width = 1920,
        .height = 1080,
        .max_fps = {
            .numerator = 10000,
            .denominator = 300000,
        },
        .exp_def = 0x0080,  // exp曝光默认值，一般应小于vts默认值，最大vts_def - 8 (曝光行，具体减去多少，参考 Sensor 手册，可能4/8/10)
        .hts_def = 0x44C * 2,// hts默认值填写对应寄存器列表中给出的默认值(可乘倍数)
        .vts_def = 0x0465,  // vts默认值填对应寄存器列表中给出的默认值
        ...
    },
    ...
};
```

6.3 `link_freq` 与 `pixel_rate` 值应该是多少

link_freq 指的是 MIPI Clk 的实际频率。注意不是 24M 的 MCLK，而是 MIPI dn/dp clk。优先通过原厂窗口查问或查找 Datasheet 是否有相关的参数。
一般情况下， link_freq 实际值不会小于如下公式的计算结果，单位是(Hz)

```
link_freq = width * height * fps * bits_per_pixel / lanes / 2
```

如果实在不知道 link_freq 的实际值，可以用示波器测量。

pixel_rate 指的是每秒传输的像素个数，在 link_freq 确定之后，可用以下公式计算：

```
pclk = link_freq * 2 * lanes / bits_per_pixel
```

6.4 怎么才算点亮 Sensor

首先需要确认是否能认到 Sensor ID，即 I2C 的读写是否有异常。这时用 media-ctl -p -d /dev/media0 应该能够看到 Sensor 的具体信息，如名称、分辨率等。如下所示：

```
# media-ctl -p -d /dev/media0 | tail -n 30
```

```
                -> "rkcif_tools_id2":0 []
        pad11: Source
                -> "stream_cif_mipi_id0":0 []
                -> "stream_cif_mipi_id1":0 []
                -> "stream_cif_mipi_id2":0 []
                -> "stream_cif_mipi_id3":0 []
                -> "rkcif_scale_ch0":0 []
                -> "rkcif_scale_ch1":0 []
                -> "rkcif_scale_ch2":0 []
                -> "rkcif_scale_ch3":0 []
                -> "rkcif_tools_id0":0 []
                -> "rkcif_tools_id1":0 []
                -> "rkcif_tools_id2":0 [ENABLED]

- entity 58: rockchip-csi2-dphy0 (2 pads, 2 links)
             type V4L2 subdev subtype Unknown flags 0
             device node name /dev/v4l-subdev1
        pad0: Sink
                [fmt:SBGGR10_1X10/1920x1080@10000/300000 field:none]
                <- "m00_b_sc200ai 4-0030":0 [ENABLED]
        pad1: Source
                -> "rockchip-mipi-csi2":0 [ENABLED]

- entity 63: m00_b_sc200ai 4-0030 (1 pad, 1 link)
             type V4L2 subdev subtype Sensor flags 0
             device node name /dev/v4l-subdev2
        pad0: Source
                [fmt:SBGGR10_1X10/1920x1080@10000/300000 field:none]
                -> "rockchip-csi2-dphy0":0 [ENABLED]
```

若 media-ctl 或 v4l2-ctl 没有编译打包在系统中，需要使用 dmesg 打印 kernel log ，查找相关 Sensor 是否 Detect 成功。

其次，上层抓图时，MIPI 要能输出数据，且不报 MIPI / ISP 相关错误，应用层能接收到帧。

6.5 Sensor 可以出流，报 CRC、ECC 等相关错误

6.5.1 确认错误来源和日志细节

定位日志：

- 首先在 `dmesg` 或 `kernel log` 中精确查找包含 `ERR1`、`ERR2`、`CRC`、`ECC`、`mipi`、`csi2`、`phy` 等关键字的错误信息。
- Rockchip 的 CSI/PHY 驱动通常在出错时会打印详细的寄存器状态。**重点记录 `ERR1`、`ERR2` 的错误类型代码** 以及 **`VC (Virtual Channel)`** 编号。
- 典型错误日志片段：

```
[  126.741234] rockchip-mipi-csi2 feaa0000.csi2-dphy: ERR1: 0x00000040 (ECC Error, VC1),
FIFO overflow? Check timings!
[  126.741259] rockchip-mipi-csi2 feaa0000.csi2-dphy: ERR2: 0x00000010 (CRC Error, VC0)
```

理解错误类型:

- **`ERR1` (CSIS_ERR1)**：常指示 **协议层错误** (ECC、帧格式错误、包头校验错误)。
- **`ERR2` (CSIS_ERR2)**：常指示 **传输层错误** (SoC 端接收 FIFO 溢出、数据收超时、同步码错误)。
- **`CRC Error`**：数据包校验和错误（传输中数据位翻转）。
- **`ECC Error`**：包头错误检测与纠正失败（通常更严重，指示物理层问题）。
- **`Overflow`**：接收端 FIFO 溢出（通常由时序不匹配引起）。

6.5.2 软件配置检查 (驱动与DTS)

Sensor 输出时序匹配：

- 检查 Sensor 驱动设置的 `HTS (Total Width)` 和 `VTS (Total Height)` **是否与 SoC 接收端 (CSI2-DPHY Controller) 配置完全一致**。不一致会导致 FIFO 溢出 (`ERR2 Overflow`)。使用以下命令验证：

```
media-ctl -p -d /dev/mediaX  # 查看 Entity 之间的链路和当前格式
v4l2-ctl -d /dev/v4l-subdevY --get-fmt-video # 查看 Sensor 输出格式
v4l2-ctl -d /dev/v4l-subdevZ --get-fmt-video # 查看 CSI-DPHY 输入格式
```

- 确保 `MIPI Pixel Clock (Sensor 输出速率)` ≤ `SoC CSI-DPHY 支持的最大速率` (RK3566: ~2.5Gbps/lane, RK3588: ~2.5Gbps/lane)。

VC (Virtual Channel) 匹配：

- 检查 Sensor 驱动中设置的 MIPI VC 号 (如 `0`, `1`) **必须与 DTS 中配置的 `csi2_dphy` 或 `rkcif_mipi_lvds` 接收 VC 号一致**。不一致会导致 CRC 错误。

DTS(Device Tree)配置：

- 核对 csi2_dphy 节点配置

```
csi2_dphy_hw: csi2-dphy@fedc0000 {
    compatible = "rockchip,rk3588-csi2-dphy-hw";
    clocks = ...; // 时钟配置必须正确
    resets = ...;
    rockchip,hw = <&csi2_dphy0>; // 硬件版本
};
csi2_dphy0: csi2-dphy0 {
    compatible = "rockchip,rk3568-csi2-dphy";
    phys = <&mipi_dphy0>; // 指向正确的PHY
    phy-names = "dphy";
    ports {
        port@0 {
            reg = <0>;
            #address-cells = <1>;
            #size-cells = <0>;
```

```
            // 关键点: CSI 接收 VC 设置
            endpoint {
                remote-endpoint = <&sensor_out>;
                data-lanes = <1 2 3 4>; // Lane数量必须匹配Sensor
                virtual-channel-index = <0>; // 必须等于Sensor输出VC
            };
        };
    };
};
```

- 检查时钟和电源
  - 确保 Sensor、MIPI DPHY、CSI2 Host Controller 的时钟频率配置正确，且无冲突。
  - Sensor 的供电（`DOVDD`, `AVDD`, `DVDD`）电压必须在规格范围内，纹波 < 50mV（用示波器测量）。

6.5.3 硬件链路排查 (物理层问题占90%以上！)

- **物理连接可靠性：**
  - **插拔排线：** 重新拔插 FPC 柔性排线，确认连接器锁定到位。**替换** 一根确认良好的 FPC 排线测试。•
  - **接触点清洁：** 检查连接器金手指是否氧化或污染，用精密电子清洁剂处理。
  - **阻抗匹配：** 确保排线阻抗为标准的 **100Ω ±10%**（差分阻抗）。劣质排线是常见错误来源。
- **信号完整性测试 (必备工具：示波器/协议分析仪):**
  - **测量电源纹波：** 用示波器测量 Sensor 所有电源引脚（特别是 1.2V/1.8V DVDD）的噪声 (<50mV pk-pk)。
  - **MIPI差分信号质量：**

  在 Sensor 端的 CLK+/CLK- 和 DATA+/DATA- 测试点上测量：
  - **眼图 (Eye Diagram)：** 眼高 ≥ 100mV，眼宽 ≥ 0.4 UI，边缘单调清晰。模糊/闭合的眼图是致命问题。
  - **斜率 (Slew Rate)：** 应在 1.5~4V/ns 范围内（依据 D-PHY 版本）。
  - **过冲/下冲 (Overshoot/Ringing)：** < 20% Vdiff pp。
  - **共模噪声 (Common Mode Noise)：** < 100mV。
  - **时序偏差 (Skew)：** Data-to-Clk Skew < 0.3 UI。
  - **低速模式验证：** 尝试将 Sensor 配置为 MIPI 低速模式（如降到 500Mbps）看错误是否消失。
- **参考时钟（MCLK）质量：**
  - 测量 Sensor 输入的 MCLK（通常是 24MHz 或 27MHz）信号：**频率稳定性 (±50ppm)，相位抖动 (< 50ps RMS)**。
  - 示波器测量要求：使用屏蔽线缆，接地环最小化。
- **节点与干扰：**
  - **地回路阻抗：** Sensor板地到SoC板地的阻抗 < 1Ω。
  - **干扰源隔离：** 远离高频电路（WiFi/DC-DC）、马达、强磁场。

6.5.4 系统级验证与调试

Lane数量和极性检查：

- 确认 DTS 中配置的 `data-lanes` 顺序和数量与实际使用的物理 Lane 一致。
- **特别注意：Lane极性翻转。** MIPI允许RX端内部翻转极性，检查DTS中是否有设置 `lane-polarities`：

降低分辨率/帧率测试：

- 将 Sensor 输出分辨率降至最低（如 320x240 @10fps），减少数据传输量，**若错误消失，则表明硬件带宽瓶颈（信号质量问题）**。

使用测试模式：

- 启用 Sensor 内建的测试图模式（Color Bar/Checkerboard），**绕开传感器模拟电路，直接测试MIPI输出**

调试工具：

- **RK ISP 调试工具：** 在 `/etc/init.d/` 中启用 `rkisp_3A_server`，查看详细日志。
- **v4l2-ctl 监控：**

```
v4l2-ctl -d /dev/video0 --stream-mmap --stream-count=100 --stream-skip=100
```

温度因素：

- 高温会劣化信号质量。观察错误率是否随温度升高而加剧（环境温度 > 85°C）。

6.5.5 紧急情况处理策略

1. **优先降速：** 立即降低 MIPI 速率至 < 1.5Gbps/lane 测试。
2. **最小化Lane：** 尝试只用 1 Lane 传输数据。
3. **隔离干扰：** 移除所有无关外设，降低 CPU/GPU 频率。
4. **替换测试：** 更换 Sensor 模块、核心板或主板（特别是怀疑PCB阻抗问题时）。

6.5.6 典型修复措施

物理层问题 (90%)：

- 更换高质量 MIPI 排线（阻抗控制 100Ω±5%）。
- 优化 PCB Layout：缩短差分线长度，消除过孔 stub，增加差分对间间距。
- 添加终端电阻 (50Ω 对地)。

软件配置问题 (9%)：

- 精确匹配时序配置：`VTS`, `HTS`, `Pixel Clock`。
- 修正 VC 索引和 Lane Map。
- 更新驱动或固件版本 (MIPI PHY 固件尤其重要)。

硬件设计缺陷 (1%)：

- 重新设计电源树，增加 LDO 滤波。
- 更换高精度晶振或时钟缓冲器。
- 提升 ESD 防护等级 (TVS 二极管选型)。

6.5.7 Sensor index注意事项

Sensor 的 index 填写于 DTS，`rockchip,camera-module-index`

为了方便设备查找，libr kaiq 及 rockit 优先支持传 Sensor 的 index 作为设备索引。tunning 工具配套的 tool_server 也是按照 Sensor index 来查找设备。如果使用类似工具没有加 -d 指定设备号，默认是0.

所以建议，

- 单摄时，sensor index 配 0 。
- 双摄时，sensor index 配 0/1。
- 多摄时，sensor index 配 0/1/2/3...

7. 可参考的文档

Sensor 驱动代码编写只是 Camera 调试中的一步，后续调试时，遇到 不同平台设备树配置、具体产品形态（快启、AOV)，或开发过程中遇到出图问题时，可参考以下开发文档：

1. Rockchip_Driver_Guide_VI_CN.pdf , last ver: v1.2.0
2. Rockchip Quick Start For Camera.pdf, last ver:v1.0.0
3. Rockchip_Trouble_Shooting_Linux5.10_Camera_CN.pdf
4. Rockchip_Developer_Guide_Linux_AOV_CN.pdf

8. 附录

8.1 CIS驱动V4L2-controls列表

CID	描述
V4L2_CID_VBLANK	Vertical blanking. The idle period after every frame during which no image data is produced. The unit of vertical blanking is a line. Every line has length of the image width plus horizontal blanking at the pixel rate defined by V4L2_CID_PIXEL_RATE control in the same sub-device.

CID	描述
V4L2_CID_HBLANK	Horizontal blanking. The idle period after every line of image data during which no image data is produced. The unit of horizontal blanking is pixels.
V4L2_CID_EXPOSURE	Determines the exposure time of the camera sensor. The exposure time is limited by the frame interval.
V4L2_CID_ANALOGUE_GAIN	Analogue gain is gain affecting all colour components in the pixel matrix. The gain operation is performed in the analogue domain before A/D conversion.
V4L2_CID_PIXEL_RATE	Pixel rate in the source pads of the subdev. This control is read-only and its unit is pixels / second. Ex mipi bus： pixel_rate = link_freq * 2 * nr_of_lanes / bits_per_sample
V4L2_CID_LINK_FREQ	Data bus frequency. Together with the media bus pixel code, bus type (clock cycles per sample), the data bus frequency defines the pixel rate (V4L2_CID_PIXEL_RATE) in the pixel array (or possibly elsewhere, if the device is not an image sensor). The frame rate can be calculated from the pixel clock, image width and height and horizontal and vertical blanking. While the pixel rate control may be defined elsewhere than in the subdev containing the pixel array, the frame rate cannot be obtained from that information. This is because only on the pixel array it can be assumed that the vertical and horizontal blanking information is exact: no other blanking is allowed in the pixel array. The selection of frame rate is performed by selecting the desired horizontal and vertical blanking. The unit of this control is Hz.

8.2 MEDIA_BUS_FMT列表

CIS Sensor类型	Sensor输出format
Bayer RAW	MEDIA_BUS_FMT_SBGGR10_1X10 MEDIA_BUS_FMT_SRGGB10_1X10 MEDIA_BUS_FMT_SGBRG10_1X10 MEDIA_BUS_FMT_SGRBG10_1X10 MEDIA_BUS_FMT_SRGGB12_1X12 MEDIA_BUS_FMT_SBGGR12_1X12 MEDIA_BUS_FMT_SGBRG12_1X12 MEDIA_BUS_FMT_SGRBG12_1X12 MEDIA_BUS_FMT_SRGGB8_1X8 MEDIA_BUS_FMT_SBGGR8_1X8 MEDIA_BUS_FMT_SGBRG8_1X8 MEDIA_BUS_FMT_SGRBG8_1X8
YUV	MEDIA_BUS_FMT_YUYV8_2X8 MEDIA_BUS_FMT_YVYU8_2X8 MEDIA_BUS_FMT_UYVY8_2X8 MEDIA_BUS_FMT_VYUY8_2X8 MEDIA_BUS_FMT_YUYV10_2X10 MEDIA_BUS_FMT_YVYU10_2X10 MEDIA_BUS_FMT_UYVY10_2X10 MEDIA_BUS_FMT_VYUY10_2X10 MEDIA_BUS_FMT_YUYV12_2X12 MEDIA_BUS_FMT_YVYU12_2X12 MEDIA_BUS_FMT_UYVY12_2X12 MEDIA_BUS_FMT_VYUY12_2X12
Only Y(黑白) 即raw bw sensor	MEDIA_BUS_FMT_Y8_1X8 MEDIA_BUS_FMT_Y10_1X10 MEDIA_BUS_FMT_Y12_1X12

8.3 Camera 参考驱动列表

CIS 数据接口	CIS 输出数据类型	Frame/Field	参考驱动
MIPI	Bayer RAW	frame	**0.3M** ov7750.c gc0403.c **0.9M** jx_h62.c jx_h63p **1.2M** ov9750.c jx-h65.c mia1321.c **2M** jx_f37.c mis20s1.c mis2031.c ov2685.c ov2680.c ov2735.c ov02g10.c ov02b10.c gc2385.c gc2355.c gc2053.c gc2093.c sc2232.c sc2239.c sc223a.c sc2310.c sc2336.c sc2336p.c sc230ai.c sc231hai.c sc235hai.c sc210iot.c sp250a.c **4M** gc4c33.c jx_k04.c jx_k17.c mis4001.c

CIS 数据接口	CIS 输出数据类型	Frame/Field	参考驱动
			mis40c1.c mis40h1.c ov4686.c ov4688.c ov4689.c os04c10.c os04a10.c sc401ai.c sc4236.c sc450ai.c sc485sl.c imx347.c 5M mis5001.c os05a20.c ov5695.c ov5648.c ov5670.c gc5024.c gc5025.c gc5035.c hm5040.c sc5239.c hynix_hi556.c 8M ar0822.c gc8613.c mis8001.c ov8858.c os08a10.c os08a20.c sc8220.c sc830ai.c sc835hai.c sc850sl.c imx378.c imx317.c imx219.c gc8034.c hynix_hi846.c s5kgm1sp.c s5k4h7yx.c 13M

CIS 数据接口	CIS 输出数据类型	Frame/Field	参考驱动
			ov13850.c ov13b10.c imx258.c ov12d2q.c
MIPI	Bayer raw hdr	frame	2M imx307.c imx327.c imx462.c gc2093.c ov02k10.c ov2718.c sc200ai.c sc2310.c jx-f37.c 4M ov4689.c os04a10.c imx347.c imx464.c sc4238.c 5M imx335.c os05a20.c sc500ai.c 8M imx334.c imx415.c
MIPI	YUV	frame	2M gc2145.c
MIPI	RAW BW	frame	0.3M ov7251.c 1M ov9281.c 1.3M sc132gs.c

CIS 数据接口	CIS 输出数据类型	Frame/Field	参考驱动
MIPI	YUV	field	tc35874x.c
ITU.BT60	Bayer RAW		**2M** imx323.c ar0230.c
ITU.BT601	YUV		**0.3M** gc0329.c gc0312.c gc032a.c **2M** gc2145.c gc2155.c gc2035.c bf3925.c
ITU.BT601	RAW BW		**0.3M** sc031gs.c **1.3M** sc032gs.c
ITU.BT656	Bayer RAW		**2M** imx323.c(可支持)